



UNIVERSITÄT  
LEIPZIG

Fakultät

Institut

Fachgebiet

*Erstbetreuer*

*Zweitbetreuer*

Thema

*Titel*

Bachelorarbeit zur Erlangung des akademischen Grades

Bachelor of Science – *Studiengang*

vorgelegt von: *Autorenname*

Matrikelnummer: *Matrikelnummer*

E-Mail-Adresse: *E-Mail*

Telefonnummer: *Telefonnummer*

Anschrift: *Straße*  
*Plz und Ort*

Leipzig, den Datum

## **Zusammenfassung**

Dein Abstract

*Schlüsselwörter:*

Deine Schlüsselwörter

**Inhaltsverzeichnis**

|                                                            |            |
|------------------------------------------------------------|------------|
| <b>Inhaltsverzeichnis</b>                                  | <b>I</b>   |
| <b>Abbildungsverzeichnis</b>                               | <b>IV</b>  |
| <b>Tabellenverzeichnis</b>                                 | <b>V</b>   |
| <b>Formelverzeichnis</b>                                   | <b>VI</b>  |
| <b>Abkürzungsverzeichnis</b>                               | <b>VII</b> |
| <b>1 Einleitung</b>                                        | <b>1</b>   |
| 1.1 Motivation und Problemstellung . . . . .               | 1          |
| 1.2 Zielstellung . . . . .                                 | 2          |
| 1.3 Aufbau der Arbeit . . . . .                            | 3          |
| <b>2 Fachliche Grundlagen</b>                              | <b>4</b>   |
| 2.1 Reinforcement Learning . . . . .                       | 4          |
| 2.1.1 Markov Decision Process . . . . .                    | 4          |
| 2.2 Strategioptimierung . . . . .                          | 5          |
| 2.2.1 Policy-Gradient-Methoden . . . . .                   | 5          |
| 2.2.2 Proximal Policy Optimization . . . . .               | 6          |
| 2.3 Convolutional Neural Networks . . . . .                | 7          |
| 2.4 Achsenparallele Begrenzungsboxen . . . . .             | 8          |
| <b>3 Stand der Forschung</b>                               | <b>9</b>   |
| 3.1 Autonomes Landen von Drohnen . . . . .                 | 9          |
| 3.2 Tiefenbildbasierte Hindernisvermeidung . . . . .       | 11         |
| 3.3 Reinforcement Learning für Drohnensteuerung . . . . .  | 12         |
| <b>4 Methodik</b>                                          | <b>16</b>  |
| 4.1 Systemüberblick . . . . .                              | 16         |
| 4.2 Formulierung als Markov-Entscheidungsprozess . . . . . | 16         |
| 4.2.1 Zustandsraum . . . . .                               | 17         |
| 4.2.2 Aktionsraum . . . . .                                | 17         |

---

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| 4.2.3    | Übergangsmodell . . . . .                                          | 18        |
| 4.2.4    | Belohnungsfunktion . . . . .                                       | 18        |
| 4.2.5    | Terminierung . . . . .                                             | 19        |
| 4.3      | Netzwerkarchitektur . . . . .                                      | 19        |
| 4.4      | Algorithmus . . . . .                                              | 19        |
| 4.4.1    | Proximal Policy Optimization . . . . .                             | 19        |
| 4.4.1.1  | Aktionsverteilung und Tanh-Squashing . . . . .                     | 20        |
| 4.4.1.2  | Hyperparameter . . . . .                                           | 20        |
| 4.5      | Trainingsumgebung . . . . .                                        | 21        |
| 4.5.1    | Physiksimulator . . . . .                                          | 21        |
| 4.5.2    | Drohnenmodell . . . . .                                            | 21        |
| 4.5.3    | Korridorgeometrie . . . . .                                        | 21        |
| 4.5.4    | Hindernisse . . . . .                                              | 21        |
| 4.6      | Kaskadierender PID-Regler . . . . .                                | 22        |
| 4.7      | Trainingsprozess . . . . .                                         | 23        |
| 4.8      | Evaluation . . . . .                                               | 23        |
| 4.8.1    | Evaluationsmetriken . . . . .                                      | 23        |
| 4.8.2    | Versuchsreihe 1: Generalisierung auf unbekannte Hindernisformen    | 24        |
| 4.8.3    | Versuchsreihe 2: Transfer in eine realistische Agrarumgebung . . . | 24        |
| <b>5</b> | <b>Ergebnisse</b>                                                  | <b>25</b> |
| 5.1      | Trainingsverhalten . . . . .                                       | 25        |
| 5.2      | Phase 1: Korridornavigation . . . . .                              | 26        |
| 5.2.1    | Nachuntersuchung: Oberflächenbasierte Kollisionserkennung . . .    | 27        |
| 5.3      | Phase 2: Weinberg-Transfer . . . . .                               | 27        |
| 5.3.1    | Sensitivitätsanalyse des Erfolgskriteriums . . . . .               | 28        |
| 5.4      | Vergleich RL vs. RRT-Connect . . . . .                             | 28        |
| <b>6</b> | <b>Diskussion</b>                                                  | <b>30</b> |
| 6.1      | Leistungsfähigkeit in der Trainingsumgebung . . . . .              | 30        |
| 6.1.1    | Trainingsverhalten und Konvergenz . . . . .                        | 30        |
| 6.1.2    | Dominante Fehlerart und Flugverhalten . . . . .                    | 30        |
| 6.2      | Transferleistung auf den Weinberg . . . . .                        | 31        |
| 6.2.1    | Leistungsabfall und Verschiebung der Fehlerarten . . . . .         | 31        |

---

|                  |                                                                       |              |
|------------------|-----------------------------------------------------------------------|--------------|
| 6.2.2            | Einordnung als Sim-to-Sim Transfer . . . . .                          | 31           |
| 6.2.3            | Perceptual Aliasing als Erklärung für die Terrain-Kollisionen . . . . | 32           |
| 6.3              | Leistungslücke zum Pfadplaner . . . . .                               | 33           |
| 6.3.1            | Informationsasymmetrie und Erfolgsrate . . . . .                      | 33           |
| 6.3.2            | Landezeit und Rechenzeit . . . . .                                    | 33           |
| 6.4              | Limitationen . . . . .                                                | 33           |
| <b>7</b>         | <b>Fazit und Ausblick</b>                                             | <b>34</b>    |
| 7.1              | Beantwortung der Forschungsfragen . . . . .                           | 34           |
| 7.2              | Perspektiven für zukünftige Forschung . . . . .                       | 34           |
| <b>Anhang</b>    |                                                                       | <b>VII</b>   |
| <b>Literatur</b> |                                                                       | <b>XIII</b>  |
| <b>Erklärung</b> |                                                                       | <b>XXIII</b> |

## Abbildungsverzeichnis

|     |                                                                                                                                                                                                                                                                                             |    |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Interaktion zwischen Agent und Umgebung in einem MDP (nach Sutton und Barto 2020, S. 48). . . . .                                                                                                                                                                                           | 4  |
| 2.2 | $L^{\text{CLIP}}$ als Funktion des Wahrscheinlichkeitsverhältnisses $r$ . Links ( $\hat{A}_t > 0$ ): der Anreiz wird bei $1 + \varepsilon$ gekappt. Rechts ( $\hat{A}_t < 0$ ): die Korrektur bleibt uneingeschränkt (nach (Schulman, Wolski u. a. 2017), Fig. 1). . . . .                  | 6  |
| 4.1 | Systemarchitektur mit zwei Steuerungsebenen: Der RL-Agent (oben) erhält Tiefenbild und Zustandsvektor, sampelt eine Aktion und gibt Sollwerte vor. Der PID-Regler (unten) setzt diese über 300 Simulationsschritte in Motordrehzahlen um. . . . .                                           | 16 |
| 4.2 | Netzwerkarchitektur: Geteilter CNN-Encoder (drei Faltungsschichten mit Batch-Normalisierung, ELU und Global Max Pooling) erzeugt einen 128-dimensionalen Merkmalsvektor, der mit dem normalisierten Zustandsvektor konkateniert und an die getrennten MLP-Köpfe weitergegeben wird. . . . . | 20 |
| 4.3 | Seitenansichten der Korridor geometrie. Links: $xz$ -Ebene (Slalom-Achse) mit schematischer Flugbahn. Rechts: $yz$ -Ebene. . . . .                                                                                                                                                          | 22 |
| 4.4 | Zielpositionen in der Weinbergumgebung . . . . .                                                                                                                                                                                                                                            | 24 |
| 5.1 | Kumulative Belohnung und Standardabweichung der Aktionsverteilung . . . . .                                                                                                                                                                                                                 | 25 |
| 5.2 | Absturzstrafe und Erfolgsbelohnung . . . . .                                                                                                                                                                                                                                                | 25 |
| 5.3 | Episodenergebnisse Phase 1 . . . . .                                                                                                                                                                                                                                                        | 26 |
| 5.4 | Exemplarische 3D-Trajektorien in Phase 1 . . . . .                                                                                                                                                                                                                                          | 27 |
| 5.5 | Episodenergebnisse Phase 2 . . . . .                                                                                                                                                                                                                                                        | 28 |
| 6.1 | Perceptual Aliasing im Tiefenbild . . . . .                                                                                                                                                                                                                                                 | 32 |

## Tabellenverzeichnis

|     |                                                                                                                                                                                                                             |    |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | Vergleich lernbasierter Systeme zur autonomen Drohnennavigation und -landung. Die Spalten <i>Hind.</i> und <i>Landung</i> kennzeichnen, ob das System aktive Hindernisvermeidung bzw. Präzisionslandung adressiert. . . . . | 14 |
| 4.1 | Komponenten des Zustandsvektors $\mathbf{o}_t \in \mathbb{R}^{17}$ . . . . .                                                                                                                                                | 17 |
| 4.2 | Gewichte der Belohnungskomponenten. Negative Gewichte kennzeichnen Strafterme. . . . .                                                                                                                                      | 19 |
| 4.3 | PPO-Hyperparameter. . . . .                                                                                                                                                                                                 | 21 |
| 4.4 | Trainingsparameter. . . . .                                                                                                                                                                                                 | 23 |
| 5.1 | Konvergenzgeschwindigkeit und Checkpoint-Selektion . . . . .                                                                                                                                                                | 26 |
| 5.2 | Vergleich AABB- und oberflächenbasierte Kollisionserkennung . . . . .                                                                                                                                                       | 27 |
| 5.3 | Sensitivitätsanalyse des Erfolgsradius in Phase 2 . . . . .                                                                                                                                                                 | 28 |
| 5.4 | Vergleich RL-Agent und RRT-Connect . . . . .                                                                                                                                                                                | 29 |

## **Formelverzeichnis**



**Abkürzungsverzeichnis**

|      |       |                                    |
|------|-------|------------------------------------|
| AABB | ..... | Axis-Aligned Bounding Box          |
| CNN  | ..... | Convolutional Neural Network       |
| GNSS | ..... | Global Navigation Satellite System |
| GPU  | ..... | Graphics Processing Unit           |
| GSO  | ..... | Google Scanned Objects             |
| HPC  | ..... | High-Performance Computing         |
| IMU  | ..... | Inertial Measurement Unit          |
| MDP  | ..... | Markov Decision Process            |
| ML   | ..... | Machine Learning                   |
| MLP  | ..... | Multilayer Perceptron              |
| PID  | ..... | Proportional-Integral-Derivative   |
| PPO  | ..... | Proximal Policy Optimization       |
| RL   | ..... | Reinforcement Learning             |
| URDF | ..... | Unified Robot Description Format   |

## 1 Einleitung

### 1.1 Motivation und Problemstellung

Die Landwirtschaft steht vor wachsenden Herausforderungen: Der Klimawandel führt zu erhöhtem Schädlingsdruck, veränderten Niederschlagsmustern und sinkenden Erträgen (Calvin u. a. 2023; Hultgren u. a. 2025; *Climate Change Trends and Impacts on California Agriculture: A Detailed Review* | MDPI 2026), während eine wachsende Weltbevölkerung und zunehmender Fachkräftemangel den Produktionsdruck zusätzlich verschärfen (Food and Agriculture Organization of the United Nations 2017; *World Population Prospects* 2026; Duckett u. a. 2018). Im Rahmen der digitalen Landwirtschaft bieten Drohnen (unbemannte Luftfahrzeuge) vielversprechende Lösungsansätze (Basso und Antle 2020): Sie ermöglichen unter anderem die gezielte Ausbringung von Pflanzenschutzmitteln sowie großflächiges Crop-Monitoring mittels Remote Sensing (Mashori u. a. 2026; Radoglou-Grammatikis u. a. 2020) und können so dazu beitragen, die Landwirtschaft effizienter und resilienter zu gestalten (Unde u. a. 2025; Guebsi u. a. 2024).

Trotz umfangreicher Forschung zu Drohnensystemen bleibt autonomes Landen eine offene Herausforderung (Amendola u. a. 2024). Unfallfreies Landen ist für die meisten Flugmissionen essentiell; gleichzeitig erfordern die vielfältigen Landeszenarien jeweils angepasste Lösungen. Bisherige Ansätze reichen von GNSS-gestützter (Global Navigation Satellite System) Positionierung (Patrik u. a. 2019) über markerbasierte visuelle Verfahren (Wubben u. a. 2019) bis hin zu markerfreien Systemen, die Landezonen eigenständig aus Sensordaten ableiten (Bartolomei u. a. 2022). Dabei zeigt sich eine wiederkehrende Einschränkung: Systeme zur Hindernisvermeidung navigieren erfolgreich durch komplexe Umgebungen, ohne gezielt zu landen (Loquercio u. a. 2021; Xu u. a. 2024), während Landesysteme überwiegend in hindernisfreien Szenarien operieren (Polvara u. a. 2020; Ladosz u. a. 2024). Die Kombination beider Teilaufgaben, Präzisionslandung mit gleichzeitiger Hindernisvermeidung, ist insbesondere in natürlichen Umgebungen mit dichter Vegetation bislang kaum untersucht. Weinberge sind ein konkretes Beispiel solcher Umgebungen: Reihenabstände von typischerweise 1,5 bis 2 m und dichtes Blattwerk erzeugen beengte Platzverhältnisse, die eine autonome Landung zwischen den Reihen erheblich erschweren.

Reinforcement Learning (RL) hat sich in jüngerer Zeit als wirksame Lösungsstrategie für Drohnenaufgaben etabliert (Kaufmann, Bauersfeld, Loquercio u. a. 2023; Hodge u. a. 2021;

Amendola u. a. 2024): Ein RL-Agent erlernt eine Steuerungsstrategie durch Interaktion mit einer simulierten Umgebung, ohne dass ein explizites Umgebungsmodell oder manuell definierte Steuerungsregeln benötigt werden. In Kombination mit Convolutional Neural Networks (CNNs) können aufgabenrelevante Merkmale direkt aus Sensordaten extrahiert werden (Mnih u. a. 2015), was eine bildbasierte Hindernisvermeidung ohne manuelle Merkmalsdefinition ermöglicht.

Da Drohnen engen Gewichts- und Energiebeschränkungen unterliegen (Semerikov u. a. 2025), beschränkt sich die vorliegende Arbeit auf eine minimale Sensorkonfiguration aus Tiefenkamera und propriozeptivem Zustandsvektor und setzt auf einen lernbasierten Ansatz, um diese begrenzte Sensorik für die autonome Landung in hindernisreichen Umgebungen auszunutzen.

## 1.2 Zielstellung

Ziel der Arbeit ist die Entwicklung eines Reinforcement-Learning-basierten Verfahrens, das eine autonome Multirotor-Drohne befähigt, mithilfe lokaler Tiefensensorik in einer hindernisreichen Umgebung sicher zu landen. Das System kombiniert einen propriozeptiven Zustandsvektor mit tiefenbildbasierter Umgebungswahrnehmung, um Hindernisse während des Landevorgangs zu erkennen und ihnen auszuweichen. Die Leistungsfähigkeit des trainierten Agenten wird anhand einer abstrakten Trainingsumgebung evaluiert und anschließend im Zero-Shot-Transfer, also ohne erneutes Training in der Zielumgebung, auf eine Umgebung mit realistischer Vegetationsstruktur getestet.

Dadurch ergeben sich folgende Fragestellungen:

### Hauptfragestellung

Inwiefern kann ein Reinforcement-Learning-basierter Ansatz eine autonome Drohne befähigen, mithilfe lokaler Tiefensensorik in hindernisreichen Umgebungen sicher zu landen?

### Nebenfragestellungen

1. Wie leistungsfähig ist der trainierte Agent in der Trainingsumgebung, und welche Faktoren begrenzen die Erfolgsrate?
2. In welchem Maß lässt sich die erlernte Navigationsleistung auf eine geometrisch verschiedenartige Umgebung mit realistischer Vegetationsstruktur übertragen?
3. Welche Leistungslücke besteht zwischen einem lernbasierten Agenten mit lokaler Sensorik und einem Pfadplaner mit vollständiger Umgebungsinformation?

### 1.3 Aufbau der Arbeit

Kapitel 2 führt die für diese Arbeit relevanten Grundlagen ein: die theoretischen Grundlagen des Reinforcement Learning von der MDP-Formulierung über Wertfunktionen bis hin zu Proximal Policy Optimization, Convolutional Neural Networks zur Merkmalsextraktion aus Bilddaten sowie achsenparallele Bounding Boxes zur Kollisionserkennung. Kapitel 3 gibt einen Überblick über bestehende Ansätze zur autonomen Drohnenlandung und Hindernisvermeidung und identifiziert die Forschungslücke, die diese Arbeit adressiert. Kapitel 4 beschreibt das entwickelte System, einschließlich der MDP-Formulierung, der CNN-MLP-Architektur, der simulierten Trainingsumgebung und des zweiphasigen Evaluationsprotokolls. Kapitel 5 präsentiert die experimentellen Ergebnisse beider Evaluationsphasen sowie den Vergleich mit einem klassischen Pfadplaner. Kapitel 6 interpretiert die Ergebnisse im Kontext der Forschungsfragen und diskutiert Limitationen des Ansatzes. Kapitel 7 fasst die zentralen Erkenntnisse zusammen und skizziert Perspektiven für weiterführende Arbeiten.

## 2 Fachliche Grundlagen

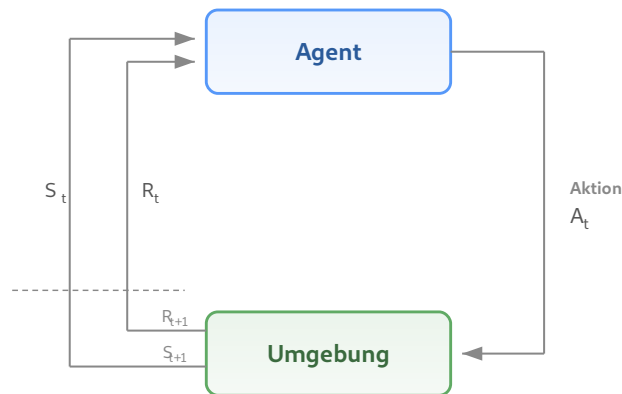
Dieses Kapitel führt die theoretischen Grundlagen ein, auf denen das entwickelte System aufbaut.

### 2.1 Reinforcement Learning

Reinforcement Learning bezeichnet das Lernen aus Interaktionen mit einer Umgebung, um ein Ziel zu erreichen. Ein Agent wählt in aufeinanderfolgenden Situationen Aktionen und erhält dafür ein skalares Belohnungssignal. Aus diesen Erfahrungen muss er eigenständig ableiten, welche Aktionen langfristig den höchsten Ertrag erzielen. Sofern nicht anders angegeben, folgt die Darstellung in diesem Abschnitt Sutton und Barto (2020).

#### 2.1.1 Markov Decision Process

Ein *Markov Decision Process* (MDP) formalisiert Reinforcement Learning Probleme als Interaktion zwischen einem **Agenten** und einer **Umgebung** in diskreten Zeitschritten  $t = 0, 1, 2, \dots$ . Der Agent beobachtet einen **Zustand**  $S_t \in \mathcal{S}$ , wählt eine **Aktion**  $A_t \in \mathcal{A}$  und erhält eine **Belohnung**  $R_{t+1} \in \mathcal{R}$  sowie einen neuen Zustand  $S_{t+1}$  (Abbildung 2.1).



**Abbildung 2.1.: Interaktion zwischen Agent und Umgebung in einem MDP (nach Sutton und Barto 2020, S. 48).**

Die Übergangsdynamik

$$p(s', r \mid s, a) = \Pr\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\} \quad (2.1)$$

charakterisiert die Umgebung vollständig. Wesentlich ist die *Markov-Eigenschaft*: Folgezustand und Belohnung hängen ausschließlich vom gegenwärtigen Zustand-Aktions-Paar ab.

Das Ziel des Agenten ist eine **optimale Strategie**  $\pi_*$ , die die kumulierte Belohnung maximiert.

## 2.2 Strategieoptimierung

Welche Verfahren zur Bestimmung einer optimalen Strategie geeignet sind, hängt maßgeblich von der Formulierung des Reinforcement-Learning-Problems ab. Die klassische Theorie liefert exakte Lösungen ausschließlich für endliche MDPs mit vollständig bekanntem Übergangsmodell; ihre Grundideen lassen sich jedoch auf komplexere Problemklassen verallgemeinern.

### 2.2.1 Policy-Gradient-Methoden

Die kumulierte Belohnung wird formal als erwarteter diskontierter Ertrag

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.2)$$

mit Diskontierungsfaktor  $\gamma \in [0, 1)$  ausgedrückt. Die Güte einer Strategie wird durch die **Zustandswertfunktion**

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s] \quad (2.3)$$

charakterisiert. Insbesondere bei hochdimensionalen Zustandsräumen und kontinuierlichen Aktionsräumen, wie sie in der Drohnensteuerung auftreten, stoßen klassische Verfahren an ihre Grenzen.

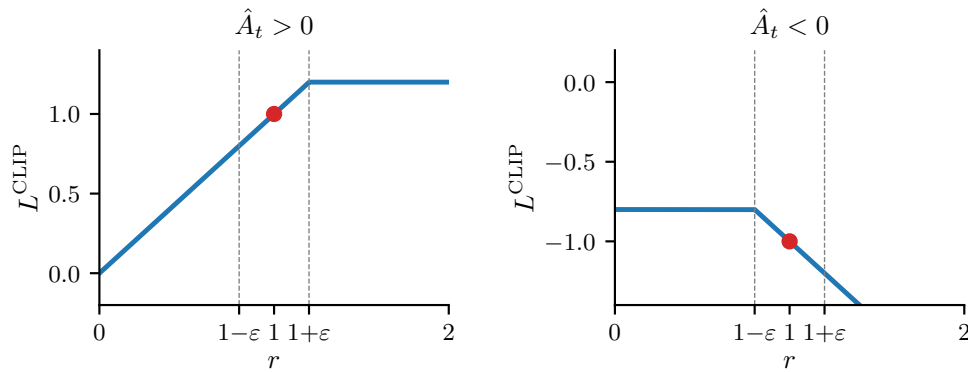
Policy-Gradient-Methoden umgehen diese Beschränkungen, indem sie die Strategie direkt als  $\pi(a \mid s, \theta)$  parametrisieren und durch Gradientenaufstieg optimieren. Dies ermöglicht sowohl die Verwendung neuronaler Netze als Funktionsapproximatoren als auch die direkte Handhabung kontinuierlicher Aktionsräume, wie sie in der Drohnensteuerung auftreten. In der Praxis wird dies als *Actor-Critic-Architektur* umgesetzt: Ein *Actor*  $\pi(a \mid s, \theta)$  repräsentiert die Strategie und wählt Aktionen; ein *Critic*  $\hat{v}(s, \mathbf{w})$  schätzt die Zustandswertfunktion und bewertet die gewählten Aktionen. Der Critic dient dabei als varianzreduzierendes Referenzniveau für den Policy-Gradienten, da der Agent nicht mehr auf den verrauschten Episodenenertrag  $G_t$  angewiesen ist, sondern die gelernte Wertschätzung als stabilere Grundlage nutzt.

### 2.2.2 Proximal Policy Optimization

Policy-Gradient-Verfahren können durch zu große Strategieänderungen pro Optimierungsschritt destabilisiert werden, da die unter der alten Strategie gesammelten Erfahrungsdaten das Verhalten der aktualisierten Strategie nicht mehr abbilden (Schulman, Wolski u. a. 2017). Trust-Region-Methoden wie TRPO (Schulman, Levine u. a. 2015) begrenzen daher die Schrittweite über eine KL-Divergenz-Schranke zwischen alter und neuer Strategie, erfordern dafür jedoch Approximationen zweiter Ordnung. *Proximal Policy Optimization* (PPO) (Schulman, Wolski u. a. 2017) erzielt eine vergleichbare Stabilisierung allein durch eine geklippte Zielfunktion:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[ \min \left( r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right], \quad (2.4)$$

wobei  $r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{alt}}}(a_t | s_t)$  das Wahrscheinlichkeitsverhältnis zwischen neuer und alter Strategie ist,  $\hat{A}_t$  der geschätzte Vorteil, berechnet mittels *Generalized Advantage Estimation* (GAE) (Schulman, Moritz u. a. 2016), und  $\varepsilon$  die Clipping-Grenze. Das Clipping bewirkt *pessimistische* Updates: Verbessert eine Strategieänderung das Objective, wird der Anreiz bei  $r_t > 1 + \varepsilon$  gekappt; verschlechtert sie es, greift die Minimumbildung und die Korrektur bleibt uneingeschränkt (Abbildung 2.2). So werden zu große Schritte in Richtung guter Erfahrungen gebremst, während schlechte Aktionen voll korrigiert werden.



**Abbildung 2.2.:  $L^{\text{CLIP}}$  als Funktion des Wahrscheinlichkeitsverhältnisses  $r$ . Links ( $\hat{A}_t > 0$ ): der Anreiz wird bei  $1 + \varepsilon$  gekappt. Rechts ( $\hat{A}_t < 0$ ): die Korrektur bleibt uneingeschränkt (nach (Schulman, Wolski u. a. 2017), Fig. 1).**

Die vollständige Verlustfunktion von PPO kombiniert das Clipping mit zwei weiteren Termen:

$$L(\theta) = L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\theta) + c_2 H[\pi_\theta](s_t), \quad (2.5)$$

wobei  $L^{VF} = (V_\theta(s_t) - V_t^{\text{targ}})^2$  der quadratische Fehler der Wertfunktionsschätzung des Critics ist und  $H[\pi_\theta]$  einen Entropie-Bonus bezeichnet, der vorzeitige Konvergenz der Aktionsverteilung verhindert.

### 2.3 Convolutional Neural Networks

**Convolutional Neural Networks** (CNNs) sind eine Klasse neuronaler Netze, die speziell für die Verarbeitung von Daten mit räumlicher Struktur entwickelt wurden, etwa Bilder in Form zweidimensionaler Pixelarrays. Sofern nicht anders angegeben, folgt die Darstellung in diesem Abschnitt LeCun u. a. (2015).

Die Architektur eines CNN besteht aus einer Kaskade von Faltungs- und Pooling-Schichten. In einer *Faltungsschicht* sind die Einheiten in sogenannten *Feature Maps* organisiert. Jede Einheit ist nur mit einem lokalen Ausschnitt der vorherigen Schicht verbunden und verrechnet diesen mit einem *Filter* (auch *Kernel*), einer kleinen quadratischen Gewichtsmatrix (Botsch 2023). Dabei werden die überdeckten Eingabewerte elementweise mit den Filtergewichten multipliziert und zu einem einzelnen Ausgabewert aufsummiert. Alle Einheiten innerhalb einer Feature Map teilen denselben Filter; mathematisch entspricht diese Operation einer diskreten Faltung. Verschiedene Feature Maps innerhalb einer Schicht verwenden unterschiedliche Filter und erkennen dadurch unterschiedliche lokale Muster. Die Filtergewichte werden nicht manuell festgelegt, sondern während des Trainings gelernt (Botsch 2023). Die gemeinsame Nutzung der Gewichte beruht auf zwei Eigenschaften natürlicher Bilddaten: Benachbarte Werte bilden häufig korrelierte lokale Muster, und diese Muster treten unabhängig von ihrer Position im Bild auf.

*Pooling-Schichten* fassen die Aktivierungen benachbarter Einheiten zusammen, typischerweise durch Auswahl des Maximums innerhalb eines lokalen Fensters (*Max-Pooling*) (Botsch 2023). Dies reduziert die räumliche Auflösung der Repräsentation und erzeugt eine gewisse Invarianz gegenüber kleinen Verschiebungen im Eingabebild.

Durch das Stapeln mehrerer Faltungs- und Pooling-Stufen entsteht eine hierarchische Merkmalsextraktion: Frühe Schichten erkennen einfache lokale Strukturen wie Kanten, mittlere Schichten kombinieren diese zu komplexeren Motiven, und tiefe Schichten repräsentieren ganze Objektteile. Die resultierende kompakte Repräsentation kann als Eingabe für nachgeschaltete Netzwerkkomponenten dienen. Mnih u. a. (2015) zeigten erstmals, dass ein CNN in dieser Rolle als Merkmalsextraktor innerhalb eines Reinforcement-Learning-Systems ein-



gesetzt werden kann, indem sie eine Strategie direkt aus hochdimensionalen Bilddaten lernen.

## 2.4 Achsenparallele Begrenzungsboxen

Eine **achsenparallele Begrenzungsbox** (*Axis-Aligned Bounding Box*, AABB) ist das kleinste achsenparallele Quader, das ein gegebenes Objekt vollständig umschließt. Sie wird durch zwei Punkte definiert: das komponentenweise Minimum und Maximum aller Objektpunkte,

$$\mathbf{p}_{\min} = (\min_i x_i, \min_i y_i, \min_i z_i), \quad \mathbf{p}_{\max} = (\max_i x_i, \max_i y_i, \max_i z_i), \quad (2.6)$$

wobei  $(x_i, y_i, z_i)$  die Eckpunkte bzw. Oberflächenpunkte des Objekts bezeichnen.

Die AABB besitzt eine *konservative* Eigenschaft: Da sie das Objekt vollständig umschließt, wird jede tatsächliche Kollision mit dem Objekt auch als Kollision mit der AABB erkannt. Im Umkehrschluss kann ein Punkt jedoch innerhalb der AABB liegen, ohne das eingeschlossene Objekt zu berühren. Der Approximationsfehler wächst mit der Abweichung der Objektgeometrie von einer quaderförmigen Gestalt; bei konkaven oder stark unregelmäßigen Formen kann die AABB ein erheblich größeres Volumen als das Objekt selbst einnehmen.

Für die Kollisionserkennung in Echtzeitsystemen bietet die AABB einen günstigen Kompromiss: Der Abstandstest reduziert sich auf sechs skalare Vergleiche pro Achse und ist damit wesentlich effizienter als Distanzberechnungen auf der tatsächlichen Objektgeometrie. Dem steht die beschriebene konservative Approximation gegenüber, die zu falsch-positiven Kollisionserkennungen führen kann.

### 3 Stand der Forschung

#### 3.1 Autonomes Landen von Drohnen

Bisherige Arbeiten zum autonomen Landen von Drohnen lassen sich in zwei grundlegende Kategorien einteilen: traditionelle und bildbasierte Methoden. Traditionelle Methoden stützen sich auf Signale externer Infrastruktur wie GNSS-Satelliten oder Funkbaken, um Position und Lage der Drohne zu bestimmen. Bildbasierte Methoden hingegen nutzen Kameradaten zur Wahrnehmung der Umgebung und lassen sich weiter unterteilen in markerbasierte Systeme, die definierte visuelle Referenzpunkte zur Lokalisierung verwenden, und markerfreie Systeme, die adäquate Landezonen eigenständig aus Bilddaten ableiten und somit keine zusätzliche Bodeninfrastruktur erfordern. Diese Einteilung orientiert sich an der Taxonomie von Arafat u. a. (2023).

Orthogonal zur Sensorik lassen sich Ansätze danach unterscheiden, welche Teilaufgaben des Guidance, Navigation and Control (GNC) (Kanellakis und Nikolakopoulos 2017) sie adressieren und wie diese zusammenwirken. Modulare Methoden zerlegen das Problem in getrennte Komponenten wie Hinderniserkennung, Lagebestimmung oder Pfadplanung, die als aufeinanderfolgende Stufen einer Pipeline gelöst werden (Kanellakis und Nikolakopoulos 2017; Wang u. a. 2024). End-to-End-Methoden hingegen bilden Sensorinput direkt auf Steuerkommandos ab, ohne solche expliziten Zwischenstufen (Wang u. a. 2024).

GNSS ist die verbreitetste Methode zur Positionsbestimmung von Drohnen (Jarraya u. a. 2025), erreicht in der Praxis jedoch nur Meter-Genauigkeit: Patrik u. a. (2019) berichten von einer Landeabweichung von 1,11 m, und auch beim Open-Source-Autopiloten PX4 beträgt die Genauigkeit mehrere Meter (Meier u. a. 2015; Wubben u. a. 2019; *Precision Landing* 2026). RTK-Korrekturen können die Genauigkeit unter idealen Bedingungen auf Zentimeter-Niveau steigern (Tavasci u. a. 2024). Dass autonomes Landen bei bekannter Relativposition grundsätzlich beherrschbar ist, zeigen Voos und Bou-Ammar (2010); die zentrale Herausforderung liegt in der Positionsgenauigkeit, nicht im Regelungsentwurf. Für Präzisionslandung zwischen Rebzeilen (Reihenabstand 1,5–2 m) reicht GNSS allein nicht aus; zudem liefert es keinerlei Umgebungswahrnehmung.

Bildbasierte Landesysteme mit visuellen Markern bieten eine deutlich höhere Präzision als rein GNSS-gestützte Methoden. So erreicht das ArUco-basierte System von Wubben u. a. (2019) durch deterministische Bildverarbeitung eine durchschnittliche Landeabweichung von

11 cm bei zuverlässiger Markerdetektion aus bis zu 30 m Höhe. Deterministische Ansätze setzen jedoch voraus, dass die Markererkennung unter allen Betriebsbedingungen robust funktioniert; bei Verdeckung, wechselnden Lichtverhältnissen oder beschädigten Markern ist diese Annahme nicht gegeben.

Reinforcement Learning bietet hier einen konzeptionellen Vorteil: Statt auf regelbasierte Detektionsalgorithmen angewiesen zu sein, lernt ein RL-Agent eine Steuerungsstrategie direkt aus Bilddaten. Polvara u. a. (2020) demonstrieren diesen Ansatz erstmals für die autonome Landung: Ihr System nutzt sequentielle Deep-Q-Networks mit diskretem Aktionsraum und einer monokularen Graustufenkamera. Ausschließlich in Simulation mit Domain Randomization trainiert, erzielt das System Erfolgsraten von 89 % für den vertikalen Abstieg, die im realen Flug auf 61 % sinken. Eine zentrale Einschränkung bleibt der diskrete Aktionsraum, der die Steuerungspräzision limitiert.

Ladosz u. a. (2024) erweitern den Ansatz auf bewegliche Landeplattformen und adressieren diese Einschränkung durch einen kontinuierlichen Aktionsraum, der erstmals auch die Vertikalgeschwindigkeit einschließt, während bei früheren Arbeiten der vertikale Abstieg typischerweise regelbasiert gesteuert wird. Der Agent erhält neben Graustufenbildern auch Daten einer inertialen Messeinheit (IMU; Geschwindigkeit und Lage) als Zustandsinformation. In der Simulation erreicht das System Erfolgsraten von bis zu 97,4 % bei einer Landeabweichung von 0,52 m; im realen Transfer sinkt die Rate auf rund 70 %. Die Autoren zeigen zudem, dass Domain Randomization zwar notwendig für den Sim-to-Real-Transfer ist, eine zu starke Randomisierung die Performance jedoch signifikant verschlechtert. Die Experimente beschränken sich auf einfache Szenarien ohne Hindernisse und gleichmäßig beleuchtete Innenräume.

Markerbasierte Systeme setzen jedoch vorplatzierte Infrastruktur voraus, und beide RL-Ansätze wurden ohne Hindernisse trainiert, wobei der Sim-to-Real-Transfer mit deutlichen Einbußen einhergeht (vgl. Polvara u. a. 2020; Ladosz u. a. 2024). Markerfreie Verfahren umgehen diese Einschränkung, indem sie Landezonen eigenständig aus Bilddaten ableiten (Yang u. a. 2018; Bartolomei u. a. 2022).

Yang u. a. (2018) verfolgen einen modularen Ansatz, bei dem aus monokularer SLAM-Rekonstruktion sichere Landezonen identifiziert werden. Das System demonstriert Landungen ohne GNSS, weist jedoch aufgrund der Pipeline-Komplexität eine Landezeit von rund zwei Minuten auf.

Bartolomei u. a. (2022) wählen einen End-to-End-Ansatz: Ihr RL-Agent bildet Tiefenbilder und semantische Segmentierungen direkt auf diskrete Steuerkommandos ab und erreicht in der Simulation Erfolgsraten von über 70 % mit lediglich einer monokularen RGB-Kamera. Im Realflug landete der Agent erfolgreich, während sowohl eine kommerzielle als auch eine State-of-the-Art-Lösung am verrauschten Sensorinput scheiterten. Einschränkend setzt das System voraus, dass semantische Segmentierungen und Tiefenbilder stets verfügbar sind, und der diskrete Aktionsraum limitiert die Steuerungsflexibilität.

Dass Deep Reinforcement Learning auch unter dynamischen Umgebungsbedingungen effektive Landesteuerungen ermöglicht, zeigen Peter u. a. (2024) mit TornadoDrone: Der TD3-Agent leitet Windeinflüsse nicht aus direkten Kraftmessungen, sondern aus indirekten Zustandsänderungen (Position, Geschwindigkeit) ab und übertrifft eine konventionelle PID-EKF-Baseline bei Landungen unter Windturbulenzen deutlich. Die realen Experimente stützen sich jedoch auf ein Vicon-Lokalisierungssystem, das unter Einsatzbedingungen nicht verfügbar ist.

### 3.2 Tiefenbildbasierte Hindernisvermeidung

Loquercio u. a. (2021) demonstrieren Hochgeschwindigkeitsnavigation durch komplexe Umgebungen mit bordeigener Sensorik. Das System nutzt Tiefenbilder einer Stereokamera und IMU-Daten, um kollisionsfreie Trajektorien zu erzeugen, die ein PID-Regler ausführt. Trainiert wird in Simulation mittels Privileged Learning mit einfachen Hindernisgeometrien; durch simuliertes Sensorrauschen gelingt ein Zero-Shot-Transfer in reale Umgebungen (wie dichte Wälder). Der Ansatz reduziert die Ausfallrate gegenüber modularen Planern um den Faktor 10, definiert Erfolg jedoch bereits bei 5 m Zielabstand.

Kim u. a. (2022) schätzen Tiefenbilder aus monokularen RGB-Aufnahmen und treffen auf dieser Basis diskrete Ausweichentscheidungen mittels Dueling-Double-DQN. Das System durchfliegt nach Zero-Shot-Transfer enge Korridore kollisionsfrei, beschränkt sich jedoch auf strukturierte Innenräume und enthält keine zielgerichtete Navigation.

Xu u. a. (2024) nutzen direkte Tiefenbilder, um mittels PPO eine kontinuierliche Strategie für sichere Navigation in Umgebungen mit statischen und dynamischen Hindernissen zu trainieren. Die Architektur verarbeitet die Tiefenbilder zu einer 3D-Belegungskarte, die zusammen mit Drohnenzustandsdaten (Position, Geschwindigkeit, Orientierung) als Eingabe für ein Actor-Critic-Netzwerk dient. Nach Zero-Shot-Transfer in eine reale Umgebung mit dynamischen Hindernissen erzielt das System die geringste Kollisionsrate im Vergleich zu

bestehenden Methoden. Auch hier beschränken sich die Testumgebungen auf strukturierte Indoor-Szenarien.

Gemeinsam zeigen diese Arbeiten, dass tiefenbildbasierte lernende Systeme Hindernisse zuverlässig vermeiden können, unabhängig davon ob die Tiefeninformation direkt gemessen (Loquercio u. a. 2021; Xu u. a. 2024) oder aus RGB-Bildern geschätzt wird (Kim u. a. 2022). Allerdings evaluiert keine der genannten Arbeiten in landwirtschaftlichen Umgebungen wie Rebzeilen oder Obstplantagen.

Vereinzelte Arbeiten adressieren die Navigation in landwirtschaftlichen Umgebungen gezielt. Wei u. a. (2025) demonstrieren ein lernbasiertes Drohnensystem für die autonome Navigation durch Obstplantagen-Reihen: Ein mittels Imitation Learning trainierter Controller steuert eine Multirotor-Drohne auf Basis einer front-montierten RGB-Kamera kollisionsfrei zwischen Baumreihen. Das System generalisiert auf ungesehene Umgebungen und kommt ohne GPS aus, beschränkt sich jedoch auf horizontale Reihennavigation. Für Weinberge zeigen Martini u. a. (2022), dass ein DRL-Agent auf Basis von Tiefenbildern ohne Positionsinformation durch simulierte Rebzeilen navigieren kann, allerdings mit einem Bodenroboter statt einer Drohne. Beide Arbeiten bestätigen, dass Vegetation in landwirtschaftlichen Umgebungen eine eigenständige Navigationsherausforderung darstellt, die durch lernbasierte Ansätze mit minimaler Sensorik adressiert werden kann. Keine der Arbeiten behandelt jedoch die vertikale Landung zwischen Vegetationsstrukturen, die eine kombinierte Hindernisvermeidung und Präzisionspositionierung erfordert.

### 3.3 Reinforcement Learning für Drohnensteuerung

Über die Wahl der Sensorik hinaus beeinflusst die Steuerungsarchitektur die Leistungsfähigkeit lernbasierter Drohnensysteme maßgeblich. Alle in den Abschnitten 3.1 und 3.2 betrachteten lernbasierten Systeme folgen einem gemeinsamen Grundmuster: Ein gelerntes Modell bildet Sensorinput auf Steuerkommandos ab, die ein nachgeschalteter Flugregler ausführt. Die Systeme unterscheiden sich jedoch in der Abstraktionsebene dieser Ausgabe: von diskreten Bewegungsprimitiven (Polvara u. a. 2020; Kim u. a. 2022; Bartolomei u. a. 2022) über kontinuierliche Geschwindigkeitskommandos (Ladosz u. a. 2024) bis hin zu Kurzzeittrajektorien, die ein expliziter PID-Regler abfliegt (Loquercio u. a. 2021).

Die Wahl der Abstraktionsebene beeinflusst die Lernperformance substantiell: Eßer u. a. (o. D.) zeigen in einer systematischen Studie über verschiedene Roboterplattformen und Aufgaben, dass der optimale Aktionsraum stark aufgabenabhängig ist und unterschiedliche Ab-

straktionsebenen zu deutlich verschiedenem Explorationsverhalten und finaler Performance führen. Speziell für Multirotor-Drohnen vergleichen Kaufmann, Bauersfeld und Scaramuzza (2022) drei Abstraktionsebenen: Geschwindigkeitskommandos, kollektiven Schub mit Drehraten sowie direkte Einzelmotorschübe. Die mittlere Ebene (Schub und Drehraten) erweist sich als beste Balance zwischen Agilität und Transferrobustheit; die höchste Abstraktion (Geschwindigkeit) schränkt die Manövrierfähigkeit ein, während die niedrigste (Einzelmotorschub) zu latenzempfindlich für den Realtransfer ist.

Eng mit der Wahl des Aktionsraums verknüpft ist die Wahl des Trainingsalgorithmus. Als Policy-Gradient-Verfahren eignet sich PPO (vgl. Abschnitt 2.2.2) besonders für kontinuierliche Aktionsräume und wird in der Drohnensteuerung häufig eingesetzt. Da Reinforcement Learning grundsätzlich große Datenmengen benötigt, hat sich GPU-parallele Simulation als effektiver Ansatz etabliert: Rudin u. a. (2022) zeigen, dass massiv-parallele Simulation mit hunderten bis tausenden Umgebungsinstanzen die Trainingszeit drastisch reduziert und schnellere Konvergenz ermöglicht. PPO eignet sich als On-Policy-Verfahren für dieses Paradigma, da es die parallel erzeugten Daten direkt verarbeitet und keinen Replay Buffer benötigt. Obwohl Off-Policy-Verfahren wie SAC eine höhere Sample-Effizienz bieten, zeigen Tayar u. a. (2025), dass PPO bei sicherheitskritischen Präzisionsaufgaben in beengten Räumen zuverlässiger konvergiert als SAC. In den in diesem Kapitel betrachteten Arbeiten setzen Bartolomei u. a. (2022), Xu u. a. (2024) und Kaufmann, Bauersfeld und Scaramuzza (2022) PPO ein; Kaufmann, Bauersfeld, Loquercio u. a. (2023) demonstrieren den bislang eindrucksvollsten Einsatz, bei dem ein ausschließlich in Simulation trainiertes System drei menschliche Weltmeister im physischen Drohnenrennen schlägt.

Die vorliegende Arbeit wählt eine noch höhere Abstraktionsebene als die von Kaufmann, Bauersfeld und Scaramuzza (2022) untersuchten: Der RL-Agent gibt eine Zielposition vor, die ein kaskadierender PID-Regler (Position  $\rightarrow$  Geschwindigkeit  $\rightarrow$  Lage  $\rightarrow$  Drehzahl) in Motorkommandos umsetzt. Diese Wahl opfert bewusst Agilität zugunsten reduzierter Lernkomplexität, da die Aufgabe Präzisionslandung statt agilen Flugs erfordert. Die Ausgabe des Agenten bleibt interpretierbar, der PID-Regler kann unabhängig vom RL-Training auf die Zielplattform abgestimmt werden, und die Lernkomplexität reduziert sich auf die Frage, *wohin* die Drohne fliegen soll.

Die Wahl der Sensorausstattung ist ein weiterer zentraler Designparameter: Drohnen sind aufgrund ihrer Energie- und Gewichtsbeschränkungen typischerweise nur mit minimaler Sensorik ausgestattet, bestehend aus Kameras und Distanzsensoren (Bartolomei u. a. 2022).

Semerikov u. a. (2025) zeigen in ihrer Übersichtsarbeit, dass Vision-Inertial-Kombinationen eine gute Balance zwischen Wahrnehmungsleistung und Systemkomplexität bieten, während leistungsfähigere Sensorsetups entsprechend größere Drohnen mit höherer Lade- und Batteriekapazität erfordern. Gleichzeitig ermöglichen moderne Algorithmen, insbesondere lernbasierte Ansätze, dass auch mit leichtgewichtiger Sensorik anspruchsvolle Aufgaben gelöst werden können, die bisher komplexere Hardware voraussetzten. Die betrachteten Arbeiten bestätigen diesen Trend: Erfolgreiche Navigation gelingt mit einer einzelnen Kamera (Kim u. a. 2022; Polvara u. a. 2020; Bartolomei u. a. 2022) oder Tiefenkamera mit Zustandsdaten (Loquercio u. a. 2021; Xu u. a. 2024). Die vorliegende Arbeit folgt diesem Prinzip: Als Eingabe dienen eine Tiefenkamera und ein propriozeptiver Zustandsvektor.

Tabelle 3.1 stellt die in diesem Kapitel betrachteten Arbeiten anhand ihrer Kernmerkmale gegenüber.

**Tabelle 3.1.: Vergleich lernbasierter Systeme zur autonomen Drohnennavigation und -landung. Die Spalten *Hind.* und *Landung* kennzeichnen, ob das System aktive Hindernisvermeidung bzw. Präzisionslandung adressiert.**

| Arbeit                            | Sensorik                 | Methode | Akt.-raum | Hind. | Umgebung        | Landung |
|-----------------------------------|--------------------------|---------|-----------|-------|-----------------|---------|
| Polvara u. a. (2020)              | Mono RGB                 | SDQN    | diskret   | –     | Indoor          | ✓       |
| Ladosz u. a. (2024)               | Mono RGB, IMU            | DrQv2   | kont.     | –     | Indoor          | ✓       |
| Yang u. a. (2018)                 | Mono RGB, Barometer      | SLAM    | –         | ✓     | Outdoor         | ✓       |
| Bartolomei u. a. (2022)           | Tiefe, Semantik          | PPO     | diskret   | ✓     | Outdoor         | ✓       |
| Peter u. a. (2024)                | IMU                      | TD3     | kont.     | –     | Indoor          | ✓       |
| Loquercio u. a. (2021)            | Stereo Tiefe, IMU        | IL      | kont.     | ✓     | Outdoor         | –       |
| Kim u. a. (2022)                  | Mono RGB, Tiefe          | D3QN    | diskret   | ✓     | Indoor/ Outdoor | –       |
| Xu u. a. (2024)                   | Mono RGB, Tiefe, Zustand | PPO     | kont.     | ✓     | Indoor          | –       |
| Wei u. a. (2025)                  | Mono RGB                 | IL      | kont.     | ✓     | Agrar           | –       |
| Martini u. a. (2022) <sup>a</sup> | Tiefe, Zustand           | SAC     | kont.     | ✓     | Agrar           | –       |
| Eigene Arbeit                     | Tiefe, Zustand           | PPO     | kont.     | ✓     | Agrar           | ✓       |

<sup>a</sup> Bodenroboter (UGV). Zustand = propriozeptiver Zustandsvektor (Position, Orientierung, Geschwindigkeit).

Zusammenfassend zeigt der Stand der Forschung zwei zentrale Lücken: Erstens werden Hindernisvermeidung und Präzisionslandung überwiegend getrennt adressiert (vgl. Tabelle 3.1): Systeme zur Hindernisvermeidung navigieren durch komplexe Umgebungen, ohne gezielt zu landen (Loquercio u. a. 2021; Kim u. a. 2022; Xu u. a. 2024), während Landesysteme überwiegend in hindernisfreien Szenarien operieren (Polvara u. a. 2020; Ladosz u. a. 2024; Peter u. a. 2024). Amendola u. a. (2024) bestätigen dieses Muster in ihrer Übersichtsarbeit: Zahlreiche RL-basierte Landesysteme reduzieren das Problem auf zwei Dimensionen mit festem vertikalem Abstieg, was die Konvergenz beschleunigt, jedoch die Strategie einschränkt und komplexere Verhaltensweisen wie Hindernisvermeidung verhindert. Zweitens fehlen Evaluationen in landwirtschaftlichen Umgebungen: Alle betrachteten Systeme testen in Innenräu-

men, Wald- oder abstrakten Outdoor-Umgebungen; die spezifischen Herausforderungen von Agrarumgebungen mit engen Vegetationsstrukturen bleiben unberücksichtigt. Zudem bleibt der Sim-to-Real-Transfer eine offene Herausforderung, insbesondere bei Nutzung visueller Sensordaten (Amendola u. a. 2024): Die Erfolgsraten sinken systematisch beim Übergang in die reale Welt (von 89 % auf 61 % (Polvara u. a. 2020), von 97,4 % auf rund 70 % (Ladosz u. a. 2024)).

Keine der betrachteten Arbeiten kombiniert Präzisionslandung, aktive Hindernisvermeidung und tiefenbildbasierte Wahrnehmung in einer Agrarumgebung (vgl. Tabelle 3.1). Die vorliegende Arbeit adressiert diese Lücken: Sie untersucht, ob ein RL-Agent mit kontinuierlichem Aktionsraum und Tiefenbildern in Simulation lernen kann, in beengten Umgebungen mit Hindernissen autonom zu landen. Ein vollständiger Sim-to-Real-Transfer liegt außerhalb des Rahmens dieser Arbeit. Um die Transferfähigkeit der gelernten Strategie dennoch zu untersuchen, wird ein Zero-Shot-Transfer in eine geometrisch verschiedenartige Simulationsumgebung (Sim-to-Sim) eingesetzt.



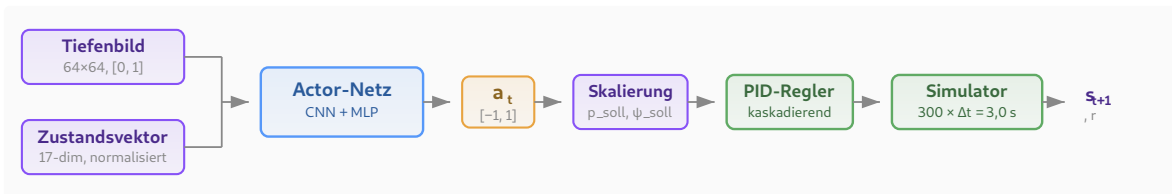
## 4 Methodik

Dieses Kapitel beschreibt das entwickelte System zur autonomen Drohnenlandung mit Hindernisvermeidung. Ein RL-Agent trifft Navigationsentscheidungen auf hoher Abstraktions-ebene, während ein kaskadierender PID-Regler diese in Motorkommandos umsetzt. Training und Evaluation finden vollständig in Simulation statt; ein Sim-to-Real-Transfer liegt außerhalb des Rahmens dieser Arbeit.

### 4.1 Systemüberblick

Das System gliedert sich in zwei Steuerungsebenen (Abbildung 4.1): Auf der oberen Ebene trifft ein RL-Agent alle 3,0 s eine Navigationsentscheidung. Auf der unteren Ebene setzt ein kaskadierender PID-Regler diese mit 100 Hz in Motordrehzahlen um. Das Entscheidungsintervall stellt sicher, dass die Drohne zwischen zwei Entscheidungen zur Ruhe kommt und das Tiefenbild aus einer stabilen Fluglage aufgenommen wird.

Der Agent erhält einen normalisierten Zustandsvektor  $\mathbf{o}_t \in \mathbb{R}^{17}$  und ein Tiefenbild  $\mathbf{D}_t \in [0, 1]^{64 \times 64}$  einer vertikal nach unten gerichteten Kamera mit 90° Sichtfeld. Ein geteilter CNN-Encoder extrahiert Hindernismerkmale aus dem Tiefenbild; der resultierende Merkmalsvektor wird mit dem Zustandsvektor konkateniert und an die MLP-Köpfe (Multilayer Perceptron) von Actor und Critic weitergegeben. Der Actor sampelt eine vierdimensionale Aktion  $\mathbf{a}_t \in [-1, 1]^4$ , die der PID-Regler über 300 Simulationsschritte in Motordrehzahlen umsetzt. Der Agent muss somit ausschließlich lernen, *wohin* die Drohne fliegen soll.



**Abbildung 4.1.: Systemarchitektur mit zwei Steuerungsebenen: Der RL-Agent (oben) erhält Tiefenbild und Zustandsvektor, sampelt eine Aktion und gibt Sollwerte vor. Der PID-Regler (unten) setzt diese über 300 Simulationsschritte in Motordrehzahlen um.**

### 4.2 Formulierung als Markov-Entscheidungsprozess

Die autonome Drohnenlandung wird als endlicher, episodischer MDP  $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$  formalisiert (vgl. Abschnitt 2.1.1), mit Diskontierungsfaktor  $\gamma = 0,99$ .

### 4.2.1 Zustandsraum

Der Zustand  $s_t \in \mathcal{S}$  setzt sich aus einem numerischen Zustandsvektor  $\mathbf{o}_t \in \mathbb{R}^{17}$  und einem Tiefenbild  $\mathbf{D}_t \in [0, 1]^{64 \times 64}$  zusammen:

$$s_t = (\mathbf{o}_t, \mathbf{D}_t). \quad (4.1)$$

Der Zustandsvektor

$$\mathbf{o}_t = (\tilde{\mathbf{p}}_{\text{rel},t}, \mathbf{q}_t, \tilde{\mathbf{v}}_t^B, \tilde{\omega}_t^B, \mathbf{a}_{t-1}) \quad (4.2)$$

enthält die in Tabelle 4.1 aufgeschlüsselten Größen. Die Tilde kennzeichnet skalierte und auf  $[-1, 1]$  geklippte Werte.

**Tabelle 4.1.: Komponenten des Zustandsvektors  $\mathbf{o}_t \in \mathbb{R}^{17}$ .**

| Größe                               | Dim. | Beschreibung                             | Skalierung                          |
|-------------------------------------|------|------------------------------------------|-------------------------------------|
| $\tilde{\mathbf{p}}_{\text{rel},t}$ | 3    | Relative Position zum Ziel               | $\times \frac{1}{15}, \in [-1, 1]$  |
| $\mathbf{q}_t$                      | 4    | Orientierungsquaternion ( $w, x, y, z$ ) | —                                   |
| $\tilde{\mathbf{v}}_t^B$            | 3    | Lineargeschwindigkeit                    | $\times 0,4, \in [-1, 1]$           |
| $\tilde{\omega}_t^B$                | 3    | Winkelgeschwindigkeit                    | $\times \frac{1}{\pi}, \in [-1, 1]$ |
| $\mathbf{a}_{t-1}$                  | 4    | Aktion des vorherigen Zeitschritts       | —                                   |

Das Tiefenbild wird von einer am Drohnenkörper vertikal nach unten gerichteten Kamera erzeugt:

$$\mathbf{D}_t = \min\left(\max\left(\frac{\mathbf{d}_{\text{roh}}}{d_{\text{max}}}, 0\right), 1\right), \quad d_{\text{max}} = 20 \text{ m}. \quad (4.3)$$

Das Tiefenbild liefert die einzige Information über Hindernisse; der Zustandsvektor enthält keine explizite Hindernisrepräsentation.

### 4.2.2 Aktionsraum

Der Aktionsraum ist vierdimensional und kontinuierlich:

$$\mathbf{a}_t = (a_x, a_y, a_z, a_\psi) \in [-1, 1]^4. \quad (4.4)$$

Die ersten drei Komponenten definieren einen Positionsversatz relativ zur aktuellen Drohnenposition:

$$\mathbf{p}_{\text{soll}} = \mathbf{p}_t + \begin{pmatrix} a_x \cdot \sigma_x \\ a_y \cdot \sigma_y \\ a_z \cdot \sigma_z \end{pmatrix}, \quad \sigma = (1,0; 1,0; 2,0) \text{ m}. \quad (4.5)$$

Die erhöhte  $z$ -Skala ermöglicht größere vertikale Schritte für den vorwiegend vertikalen Abstieg. Die vierte Komponente legt den Sollgierwinkel fest:  $\psi_{\text{soll}} = a_\psi \cdot 180^\circ$ . Die Sollwerte werden nicht direkt an die Motoren übergeben, sondern von einem kaskadierenden PID-Regler in Motordrehzahlen umgesetzt (vgl. Abschnitt 4.6).

### 4.2.3 Übergangsmodell

Die Übergangsdynamik wird deterministisch durch den Physiksimulator bestimmt (Schrittweite  $\Delta t = 0,01$  s, 100 Hz). Der RL-Agent trifft alle 300 Schritte eine Entscheidung (Entscheidungsintervall 3,0 s); dazwischen steuert der PID-Regler die Drohne zum Sollzustand.

### 4.2.4 Belohnungsfunktion

Die Belohnungsfunktion kombiniert dichte Signale, die den Fortschritt zur Zielerreichung messen, mit spärlichen terminalen Signalen (Polvara u. a. 2020). Die Fortschrittskomponente ist als potentialbasiertes Reward Shaping im Sinne von Ng u. a. (1999) konstruiert; die übrigen dichten Komponenten weichen bewusst davon ab. Die Belohnungsfunktion setzt sich als gewichtete Summe aus fünf Komponenten zusammen:

$$r_t = w_{\text{prog}} r_{\text{prog}} + w_{\text{nah}} r_{\text{nah}} + w_{\text{prox}} r_{\text{prox}} + w_{\text{crash}} r_{\text{crash}} + w_{\text{erf}} r_{\text{erf}} \quad (4.6)$$

Dabei bezeichnet  $d_t = \|\mathbf{p}_{\text{Ziel}} - \mathbf{p}_t\|$  den euklidischen Abstand zum Ziel.

**Fortschritt.**  $r_{\text{prog}} = d_t - d_{t+1}$  misst die Annäherung an das Ziel pro Zeitschritt.

**Nähe.**  $r_{\text{nah}} = \exp(-d_{t+1})$  liefert ein exponentiell ansteigendes Signal in Zielnähe und belohnt das Verbleiben nahe dem Ziel.

**Hindernisproximität.**

$$r_{\text{prox}} = \max\left(1 - \frac{d_{\min}}{d_s}, 0\right), \quad d_s = 3,0 \text{ m} \quad (4.7)$$

bestraft das Eindringen in einen Sicherheitsradius um Hindernisse, wobei  $d_{\min}$  den minimalen AABB-Abstand bezeichnet.

**Absturz.**  $r_{\text{crash}} = 1$  bei Verletzung einer Abbruchbedingung (vgl. Abschnitt 4.2.5).

**Erfolg.**  $r_{\text{erf}} = 1$  bei Erreichen des Landeziels.

Die Gewichte (Tabelle 4.2) wurden experimentell abgestimmt. Eine zu hohe Gewichtung der Strafterme führte in Vorversuchen zu stagnierendem Schwebeverhalten; eine Timeout-

Strafe wurde bewusst ausgelassen, da der Agent ohne Zeitbeobachtung im Zustandsvektor den Episodenabbruch nicht antizipieren kann (Rudin u. a. 2022).

**Tabelle 4.2.: Gewichte der Belohnungskomponenten. Negative Gewichte kennzeichnen Strafterme.**

| Komponente          | Symbol             | Gewicht |
|---------------------|--------------------|---------|
| Fortschritt         | $w_{\text{prog}}$  | 1,0     |
| Nähe                | $w_{\text{nah}}$   | 2,0     |
| Hindernisproximität | $w_{\text{prox}}$  | -0,2    |
| Absturz             | $w_{\text{crash}}$ | -10,0   |
| Erfolg              | $w_{\text{erf}}$   | 20,0    |

#### 4.2.5 Terminierung

Eine Episode endet bei **Erfolg** (Drohne verbleibt 3,0 s innerhalb von 0,5 m um das Landeziel), **Absturz** (Flughöhe  $z < 0,2$  m, Neigung  $> 60^\circ$ , AABB-Abstand  $< 0,3$  m oder Verlassen des Korridors) oder **Timeout** ( $T_{\text{max}} = 20$  Entscheidungsschritte, 60 s). Timeouts werden im Vorteilsschätzer als nicht-terminale Übergänge behandelt.

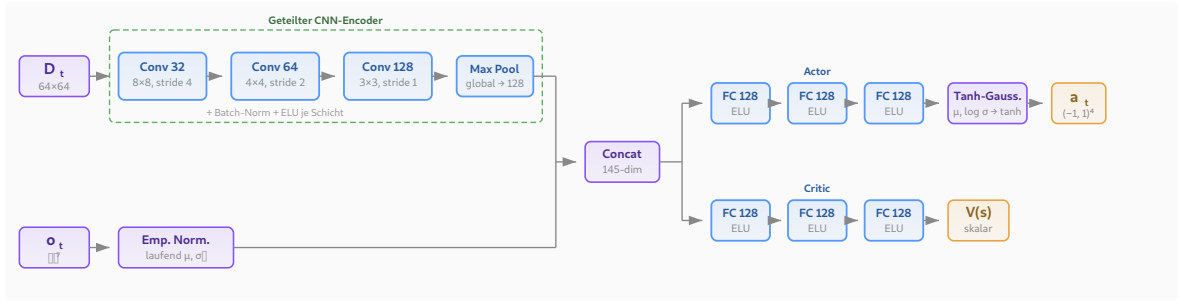
### 4.3 Netzwerkarchitektur

PPO verwendet eine Actor-Critic-Architektur (vgl. Abschnitt 2.2.2). Der CNN-Encoder besteht aus drei Faltungsschichten mit aufsteigender Kanalzahl (32, 64, 128), abnehmenden Kernelgrößen ( $8 \times 8$ ,  $4 \times 4$ ,  $3 \times 3$ ) und Schrittweiten (4, 2, 1), jeweils gefolgt von Batch-Normalisierung und ELU-Aktivierung. Ein abschließendes Global Max Pooling erzeugt einen 128-dimensionalen Merkmalsvektor. Der Zustandsvektor durchläuft eine laufende empirische Normalisierung. Der resultierende Vektor wird an die MLP-Köpfe weitergegeben: Actor mit drei Schichten à 64 Neuronen, Critic mit zwei Schichten à 32 Neuronen (beide ELU). Actor und Critic teilen sich den CNN-Encoder. Der Actor gibt pro Aktionsdimension  $\mu$  und  $\log \sigma$  aus und parametrisiert eine Tanh-Gaußverteilung (Abschnitt 4.4.1.1); der Critic gibt einen skalaren State-Value aus. Abbildung 4.2 zeigt die vollständige Architektur.

### 4.4 Algorithmus

#### 4.4.1 Proximal Policy Optimization

Als Trainingsalgorithmus wird PPO (Schulman, Wolski u. a. 2017) eingesetzt (vgl. Abschnitt 2.2.2). PPO hat sich als Standardverfahren für kontinuierliche Drohnensteuerung etabliert (Bartolomei u. a. 2022; Xu u. a. 2024; Kaufmann, Bauersfeld, Loquercio u. a. 2023)



**Abbildung 4.2.: Netzwerkarchitektur: Geteilter CNN-Encoder (drei Faltungsschichten mit Batch-Normalisierung, ELU und Global Max Pooling) erzeugt einen 128-dimensionalen Merkmalsvektor, der mit dem normalisierten Zustandsvektor konkateniert und an die getrennten MLP-Köpfe weitergegeben wird.**

und eignet sich als On-Policy-Verfahren für GPU-parallele (Graphics Processing Unit) Simulation, da es die parallel erzeugten Daten direkt verarbeitet und keinen Replay Buffer benötigt (Rudin u. a. 2022). Neuere Varianten wie Phasic Policy Gradient (Cobbe u. a. 2021) wurden nicht evaluiert, da die verwendete Trainingsumgebung rsl-rl ausschließlich Standard-PPO bereitstellt.

#### 4.4.1.1 Aktionsverteilung und Tanh-Squashing

Der Aktionsraum ist auf  $[-1, 1]^4$  beschränkt. Ein naives Clipping würde die Log-Wahrscheinlichkeit auf dem unbeschnittenen Rohwert berechnen, was zu verzerrten Gradienten führt, da PPO das Wahrscheinlichkeitsverhältnis  $r_t(\theta)$  nutzt (vgl. Abschnitt 2.2.2). Statt Clipping wird eine invertierbare Transformation (Haarnoja u. a. 2018) verwendet:

$$a_i = \tanh(u_i), \quad u_i \sim \mathcal{N}(\mu_i, \sigma_i^2). \quad (4.8)$$

Die zugehörige Log-Wahrscheinlichkeit ergibt sich über die Substitutionsregel:

$$\log \pi(\mathbf{a} \mid s) = \log \mathcal{N}(\mathbf{u} \mid \boldsymbol{\mu}, \boldsymbol{\sigma}^2) - \sum_{i=1}^4 \log(1 - \tanh^2(u_i)). \quad (4.9)$$

Zusätzlich erzwingt ein Floor  $\log \sigma_i \geq -2,0$  ( $\sigma_i \geq 0,135$ ) eine Mindestexploration.

#### 4.4.1.2 Hyperparameter

Tabelle 4.3 fasst die PPO-Hyperparameter zusammen. Die Werte für  $\varepsilon$ ,  $\gamma$ ,  $\lambda$  und  $K$  entsprechen den Standardwerten (Schulman, Wolski u. a. 2017; Schulman, Moritz u. a. 2016). Der Entropie-Koeffizient  $c_2 = 0,003$  wurde niedrig gewählt, da die Tanh-Verteilung empfindlich auf hohe Entropie-Anreize reagiert.

**Tabelle 4.3.: PPO-Hyperparameter.**

| Parameter              | Wert  |
|------------------------|-------|
| Clipping $\varepsilon$ | 0,2   |
| Diskontierung $\gamma$ | 0,99  |
| GAE $\lambda$          | 0,95  |
| Lern-Epochen $K$       | 5     |
| Mini-Batches $M$       | 4     |
| $c_1$ (Value-Loss)     | 1,0   |
| $c_2$ (Entropie)       | 0,003 |
| Gradient-Clipping      | 1,0   |

## 4.5 Trainingsumgebung

### 4.5.1 Physiksimulator

Die vorliegende Arbeit verwendet Genesis (Genesis Authors 2024), einen quelloffenen, GPU-parallelisierten Physiksimulator für Robotik. Genesis führt bis zu 4096 Umgebungen GPU-parallel aus, stellt Tiefenkameras bereit und importiert beliebige 3D-Meshes. Gegenüber Gazebo (Koenig und Howard 2004) und PyBullet (Coumans und Bai 2016–2021) bietet Genesis GPU-Parallelisierung; gegenüber dem proprietären Isaac Gym (Makoviychuk u. a. 2021) eine geringere Einstiegskomplexität.

### 4.5.2 Drohnenmodell

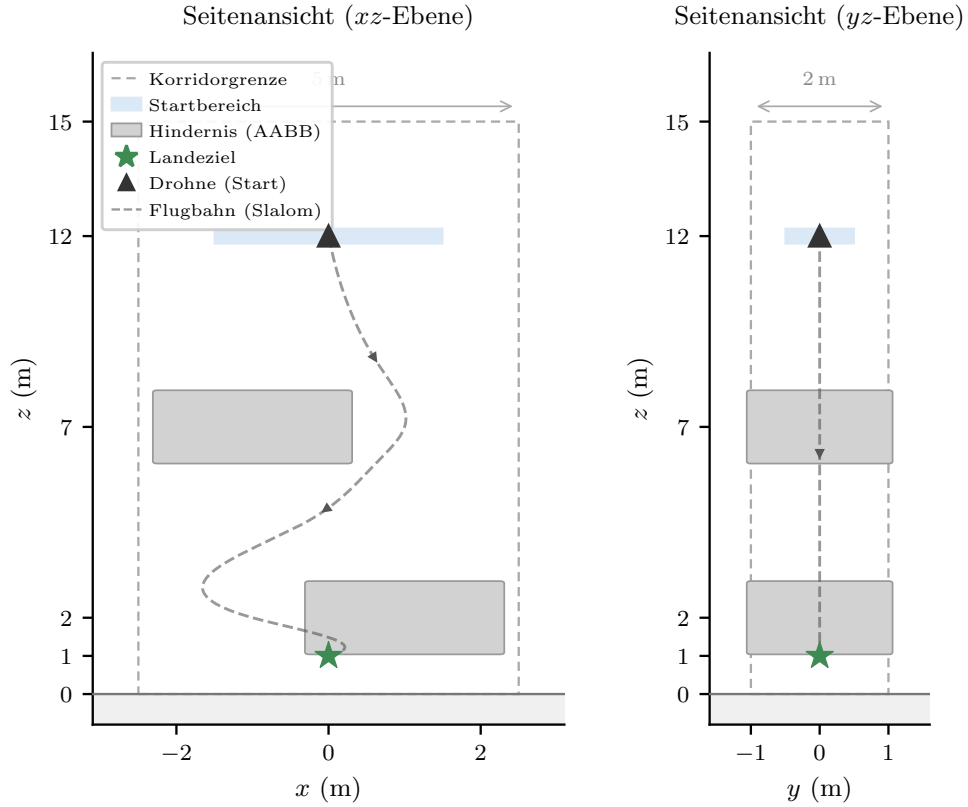
Die Drohne wird als URDF-Modell (Unified Robot Description Format) mit vier Rotoren simuliert (Masse 0,714 kg, Schub-Gewicht-Verhältnis 2,25, Hover-Drehzahl 1789 RPM, maximale Drehzahl 2700 RPM).

### 4.5.3 Korridorgeometrie

Die Trainingsumgebung bildet einen vertikalen Korridor der Ausdehnung  $5 \times 2 \times 15$  m ( $x \times y \times z$ ) ab (Abbildung 4.3). Der Korridor besitzt keine physischen Wände; das Verlassen des Begrenzungsrahmens wird als Absturz gewertet. Die Drohne wird bei  $z = 12$  m mit zufälliger horizontaler Position innerhalb von  $\pm 1,5$  m ( $x$ ) bzw.  $\pm 0,5$  m ( $y$ ) initialisiert. Das Landeziel ist fixiert bei  $\mathbf{p}_{\text{ziel}} = (0, 0, 1)^\top$  m. Während des Abstiegs durchquert die Drohne zwei horizontale Hindernisschichten bei  $z \approx 2$  m und  $z \approx 7$  m.

### 4.5.4 Hindernisse

Als Hindernisse dienen sechs 3D-Objekte aus dem Google Scanned Objects Datensatz (GSO) (Downs u. a. 2022), jeweils um den Faktor 21 bis 63 skaliert, sodass ihre horizontale Ausdehnung



**Abbildung 4.3.: Seitenansichten der Korridorgeometrie. Links:  $xz$ -Ebene (Slalom-Achse) mit schematischer Flugbahn. Rechts:  $yz$ -Ebene.**

die gesamte  $y$ -Breite des Korridors abdeckt. Für die Kollisionserkennung und die Hindernisproximität (vgl. Abschnitt 4.2.4) wird die achsenparallele Begrenzungsbox (AABB) jedes Objekts herangezogen. Diese konservative Approximation umschließt das Mesh vollständig, sodass die Drohne bereits vor Berührung der tatsächlichen Oberfläche als kollidiert gilt. Die Platzierung folgt einem erzwungenen Slalom entlang der  $x$ -Achse: Pro Hindernisschicht wird ein zufällig gewähltes Objekt auf einer Seite der Korridormitte positioniert (0,6 bis 1,2 m Versatz); aufeinanderfolgende Schichten wechseln die Seite.

#### 4.6 Kaskadierender PID-Regler

Der RL-Agent gibt Sollposition und Sollgierwinkel vor; ein dreistufiger kaskadierender PID-Regler (Proportional-Integral-Derivative; Position  $\rightarrow$  Geschwindigkeit  $\rightarrow$  Lage) setzt diese in Motordrehzahlen um. Die Verstärkungen sind in Tabelle E1 im Anhang aufgeführt. Der Regler wurde eigens für diese Simulationsumgebung entwickelt, da ein etablierter Flugreglerstack wie PX4 im verwendeten Simulator nicht zur Verfügung stand.

## 4.7 Trainingsprozess

Tabelle 4.4 fasst die Trainingsparameter zusammen. Das Training wird auf dem HPC-Cluster (High-Performance Computing) der Universität Leipzig mit einer NVIDIA A30 (Ampere-Architektur, 24 GB VRAM) durchgeführt, die den für die Tiefenkamera benötigten Batch-Renderer unterstützt (Compute Capability  $\geq 7.5$ ). Die Rollout-Länge  $T = 60$  ergibt bei 3,0 s Entscheidungsintervall eine Dauer von 180 s (drei vollständige Episoden). Zu Beginn werden die Episodenlängen zufällig initialisiert, um korrelierte Transitionsblöcke zu vermeiden (Rudin u. a. 2022).

**Tabelle 4.4.: Trainingsparameter.**

| Parameter                 | Wert                          |
|---------------------------|-------------------------------|
| Parallele Umgebungen $N$  | 512                           |
| Schritte pro Umgebung $T$ | 60                            |
| Batch-Größe $N \times T$  | 30 720                        |
| Lernrate $\alpha$         | $1 \times 10^{-3}$ (konstant) |
| Optimizer                 | Adam                          |
| Trainingsiterationen      | 1 000                         |
| GPU                       | NVIDIA A30 (24 GB)            |
| Laufzeit                  | $\sim 24$ h                   |
| Checkpoint-Intervall      | 100 Iterationen               |

## 4.8 Evaluation

Die Evaluation untersucht die trainierte Strategie anhand zweier Versuchsreihen mit jeweils 1 000 Episoden und vergleicht sie mit einem RRT-Connect-Pfadplaner.

### 4.8.1 Evaluationsmetriken

Alle Metriken werden mit 95 %-Bootstrap-Konfidenzintervallen ( $B = 10\,000$ , Perzentilmethode) (Efron und Tibshirani 1994) versehen. Erhoben werden: **Erfolgsrate**, **Fehlerartenverteilung** (Hinderniskollision, Bodenkollision, Korridorverlassen, Timeout), **Landezeit** (Median mit IQR, nur erfolgreiche Episoden), **minimaler Hindernisabstand** (AABB-basiert, Mittelwert und globales Minimum), **Pfادهffizienz**

$$\eta_{\text{Pfad}} = \frac{\sum_{t=0}^{T-1} \|\mathbf{p}_{t+1} - \mathbf{p}_t\|}{\|\mathbf{p}_{\text{Ziel}} - \mathbf{p}_0\|} \quad (4.10)$$

(Median mit IQR, nur erfolgreiche Episoden) und **Rechenzeit pro Episode** (Median mit IQR).

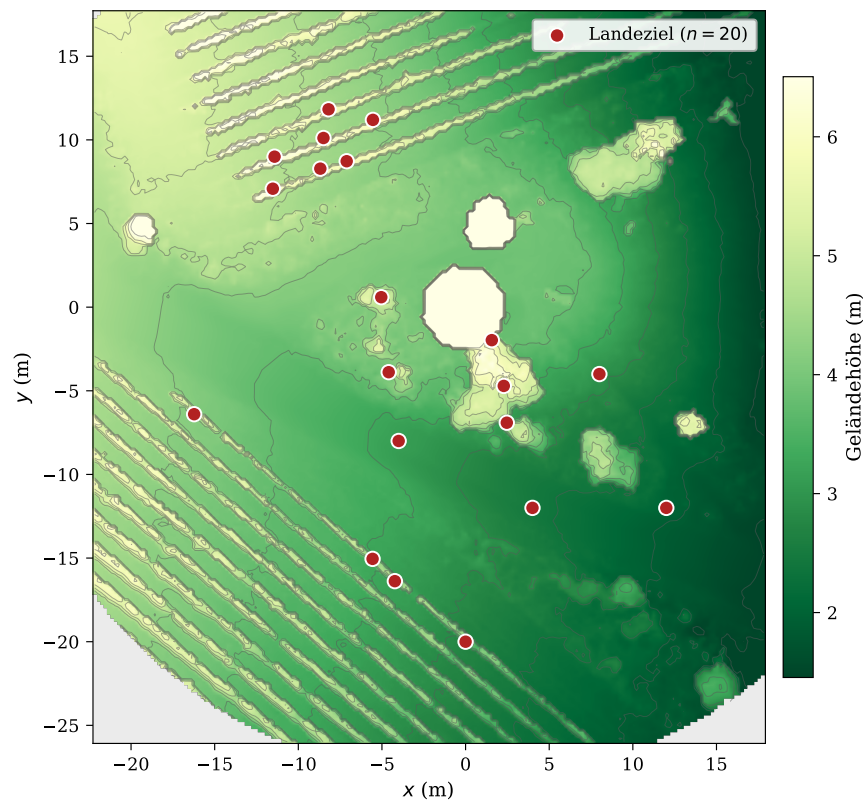


### 4.8.2 Versuchsreihe 1: Generalisierung auf unbekannte Hindernisformen

Die erste Versuchsreihe prüft die Generalisierung auf zuvor ungesehene Hindernisformen: Die sechs Trainingsobjekte werden durch sechs neue GSO-Objekte gleicher Skalierung ersetzt; alle übrigen Parameter bleiben identisch. Als Vergleichsverfahren dient ein RRT-Connect-Pfadplaner, der auf der vollständigen Kollisionsgeometrie operiert. Die Leistungsdifferenz quantifiziert den Preis eingeschränkter Sensorik, da der RL-Agent ausschließlich ein lokales Tiefenbild und den propriozeptiven Zustandsvektor erhält.

### 4.8.3 Versuchsreihe 2: Transfer in eine realistische Agrarumgebung

Die zweite Versuchsreihe untersucht den Zero-Shot-Transfer in eine photogrammetrisch erfasste Weinbergumgebung (Eltville, Deutschland, vierfache Skalierung). Die Kollisionserkennung erfolgt über ein vorberechnetes Höhenraster mit einem Sicherheitsabstand von 0,5 m; die AABB-basierte Hinderniskollision und die Korridorgrenzenprüfung entfallen. Als Landeziele dienen 20 vordefinierte Positionen (15 zwischen Reihen, 5 auf offenem Gelände); die Drohne wird in variabler Höhe (10 bis 15 m) über dem Ziel initialisiert. Abbildung 4.4 zeigt die Zielpositionen.

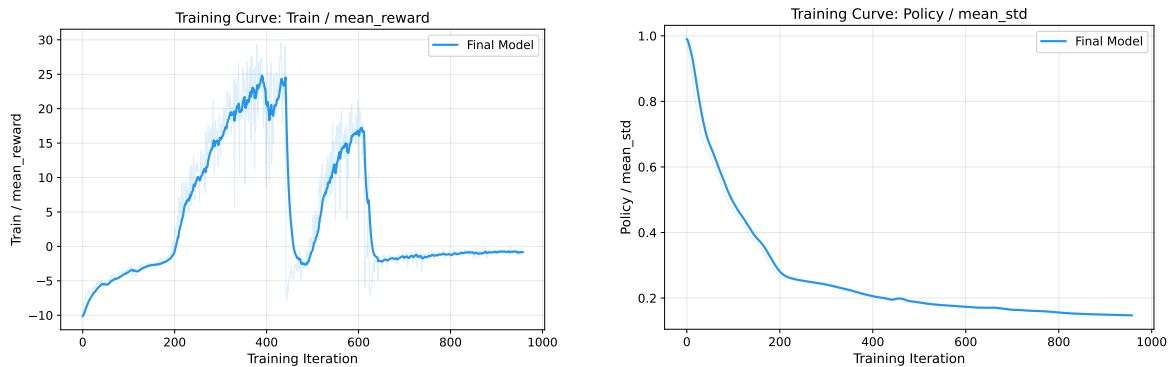


**Abbildung 4.4.: Draufsicht auf das Höhenraster der Weinbergumgebung mit den 20 vordefinierten Zielpositionen. Die diagonalen Strukturen entsprechen den Reihen.**

## 5 Ergebnisse

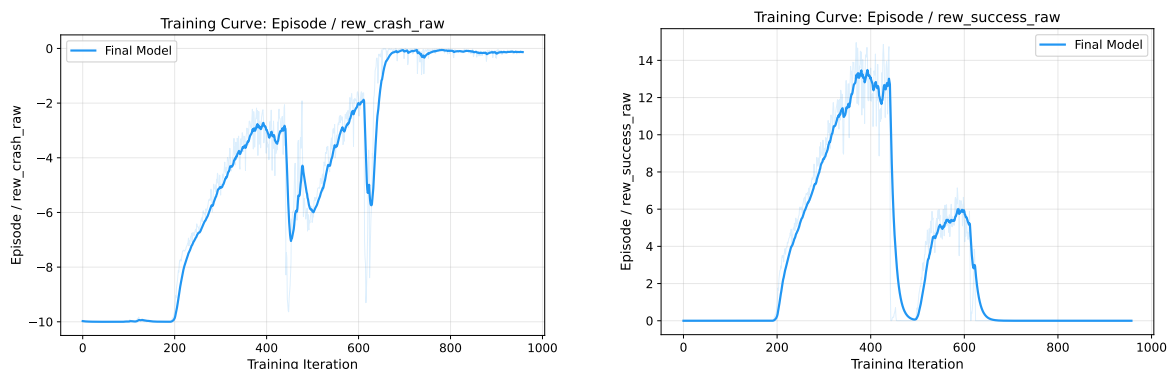
### 5.1 Trainingsverhalten

Das Training wurde mit 512 parallelen Umgebungen auf einer NVIDIA A30 GPU durchgeführt (341 FPS Durchsatz) und nach 957 Iterationen durch die 24-Stunden-GPU-Zeitbegrenzung beendet. Abbildung 5.1 zeigt den Belohnungsverlauf.



**Abbildung 5.1.: Kumulative Belohnung (links) und mittlere Standardabweichung der Aktionsverteilung (rechts) über 957 Trainingsiterationen. Geglättet mit EMA  $\alpha = 0,9$ , transparenter Hintergrund: Rohwerte.**

Der Trainingsverlauf lässt sich in zwei Phasen unterteilen. In der ersten Phase (Iteration 0 bis 441) steigt die kumulative Belohnung kontinuierlich an. Der Agent lernt zunächst die Zielnavigation; ab circa Iteration 200 erscheint erstmals die Erfolgsbelohnung (Abbildung 5.2), und gleichzeitig sinkt die Absturzrate. Die kumulative Belohnung erreicht ihren Höchstwert von 24,8 bei Iteration 390. Die Standardabweichung der Aktionsverteilung fällt monoton von 0,99 auf 0,15, was die Konvergenz zu einer deterministischen Strategie zeigt. Die übrigen Belohnungskomponenten bestätigen den Lernfortschritt (Abbildungen A1 und A2 im Anhang).



**Abbildung 5.2.: Unskalierte Absturzstrafe (links) und Erfolgsbelohnung (rechts). Geglättet mit EMA  $\alpha = 0,9$ .**

Ab Iteration 442 bricht die kumulative Belohnung abrupt ein: Innerhalb einer einzigen Iteration fällt sie von 29,2 auf  $-6,1$ . Die Standardabweichung bleibt nahezu unverändert ( $0,193 \rightarrow 0,194$ ); der Kollaps betrifft die Mittelwerte der Strategie, nicht die Explorationsbreite.

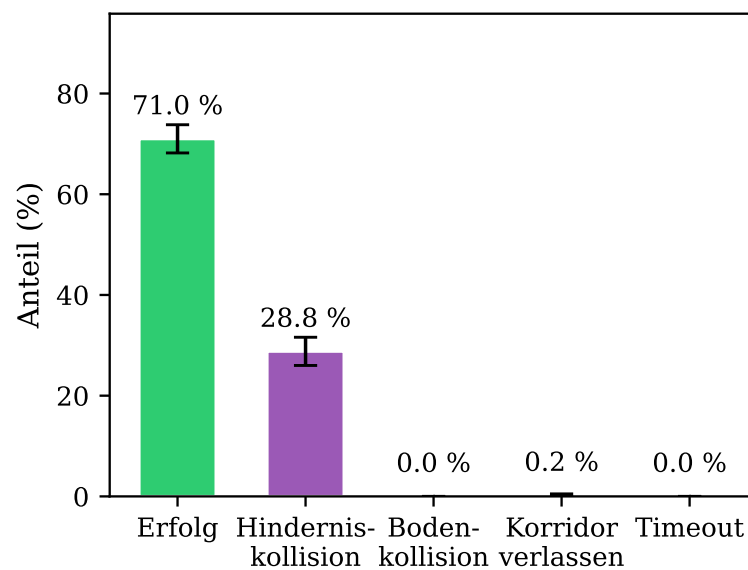
Der beste Checkpoint liegt bei Iteration 400 (Tabelle 5.1). Auf Basis des Belohnungsverlaufs wurde dieser für alle Evaluierungen ausgewählt (*Early Stopping*).

**Tabelle 5.1.: Konvergenzgeschwindigkeit und Checkpoint-Selektion (512 Umgebungen, NVIDIA A30, 341 FPS).**

| Checkpoint    | Iteration | Laufzeit | Kum. Belohnung |
|---------------|-----------|----------|----------------|
| Bestes Modell | 400       | 10,4 h   | 23,0           |
| Post-Kollaps  | 900       | 22,6 h   | $-1,1$         |
| Trainingsende | 957       | 24,0 h   | $-0,8$         |

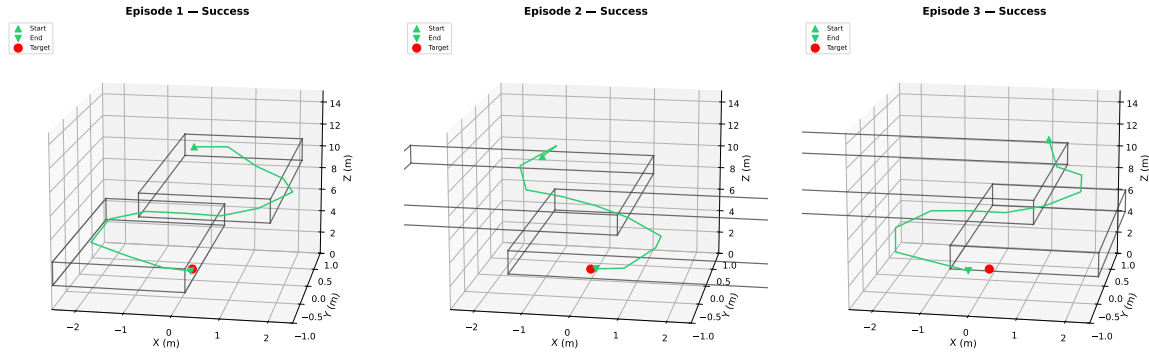
## 5.2 Phase 1: Korridornavigation

Versuchsreihe 1 (vgl. Abschnitt 4.8.2) evaluiert die trainierte Strategie (Checkpoint 400) über 1 000 Episoden mit zuvor ungesehenen Hindernisobjekten. Der Agent erreicht eine Erfolgsrate von 71,0 % (Abbildung 5.3). Die fehlgeschlagenen Episoden verteilen sich nahezu vollständig auf Hinderniskollisionen (28,8 %); die verbleibenden 0,2 % enden durch Verlassen des Korridors. Bodenkollisionen und Timeouts treten nicht auf. Tabelle B1 im Anhang listet alle Metriken mit Konfidenzintervallen.



**Abbildung 5.3.: Episodenergebnisse der Phase-1-Evaluation (1 000 Episoden). Fehlerbalken zeigen 95 %-Bootstrap-Konfidenzintervalle.**

Das Flugverhalten zeigt zwei wiederkehrende Muster (Abbildung 5.4): Bei freier Bahn vollzieht der Agent einen annähernd vertikalen Abstieg (mediane Pfadeffizienz 1,19). In der Nähe eines Hindernisses bremst er spät ab und weicht lateral mit geringem Sicherheitsabstand aus (mittlerer Minimalabstand 0,52 m).



**Abbildung 5.4.: Exemplarische 3D-Trajektorien dreier erfolgreicher Episoden. Graue Quader: AABB der Hindernisse. Grünes Dreieck: Startposition, roter Punkt: Landeziel.**

### 5.2.1 Nachuntersuchung: Oberflächenbasierte Kollisionserkennung

Das knappe Passieren bei hoher Kollisionsrate legte nahe, dass die konservative AABB-Erkennung einen Teil der Fehlschläge verursacht. In einer Nachuntersuchung mit 1 000 Episoden wurde die AABB-Kollisionserkennung durch die geometrisch exakte Oberflächenkollisionsprüfung des Simulators ersetzt.

Mit oberflächenbasierter Kontakterkennung steigt die Erfolgsrate auf 98,4 % (Tabelle 5.2). Ein erheblicher Anteil der AABB-Kollisionen sind demnach Fehlalarme: Die Drohne durchfliegt die Begrenzungsbox, ohne die Oberfläche zu berühren. Tabelle C1 im Anhang listet alle Metriken.

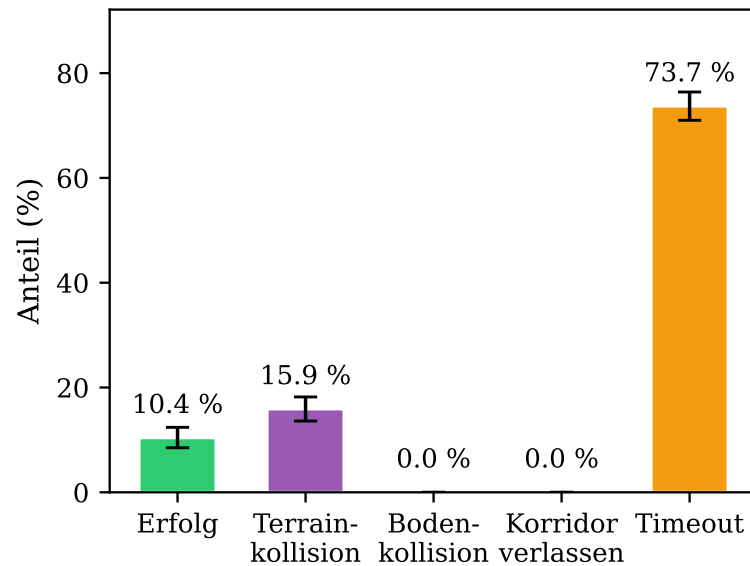
**Tabelle 5.2.: Vergleich AABB-Distanz und oberflächenbasierte Kontakterkennung ( $n = 1\,000$ ).**

| Metrik                  | AABB   | Oberfläche |
|-------------------------|--------|------------|
| Erfolgsrate             | 71,0 % | 98,4 %     |
| Hinderniskollision      | 28,8 % | 1,2 %      |
| Verlassen des Korridors | 0,2 %  | 0,2 %      |
| Timeout                 | 0,0 %  | 0,2 %      |

### 5.3 Phase 2: Weinberg-Transfer

Versuchsreihe 2 (vgl. Abschnitt 4.8.3) evaluiert den Zero-Shot-Transfer in die Weinbergumgebung über 1 000 Episoden. Der Agent erreicht eine Erfolgsrate von 10,4 % (Abbildung 5.5).

Timeouts dominieren mit 73,7 %; Terrain-Kollisionen verursachen 15,9 %. Der mediane Abstand zum Landeziel am Episodenende beträgt 1,19 m; 59,7 % der Timeout-Episoden befinden sich innerhalb von 1,5 m zum Ziel. Tabelle D1 im Anhang listet alle Metriken.



**Abbildung 5.5.: Episodenergebnisse der Phase-2-Evaluation (1 000 Episoden, Zero-Shot-Transfer). Fehlerbalken zeigen 95 %-Bootstrap-Konfidenzintervalle.**

In der Mehrzahl der Fälle erreicht der Agent die Umgebung des Landeziels, verbleibt jedoch außerhalb des Erfolgsradius von 0,5 m mit ineffektiven Korrekturbewegungen.

### 5.3.1 Sensitivitätsanalyse des Erfolgskriteriums

Mit einem erweiterten Erfolgsradius von 1,5 m steigt die Erfolgsrate auf 69,7 % (Tabelle 5.3). Die Terrain-Kollisionsrate sinkt von 15,9 % auf 9,4 %, da ein Teil der zuvor als Timeout gewerteten Episoden nun frühzeitig als Erfolg beendet wird. Der Timeout-Anteil sinkt von 73,7 % auf 27,7 %.

**Tabelle 5.3.: Sensitivitätsanalyse des Erfolgsradius  $r$  in der Weinbergumgebung ( $n = 1\,000$  je Bedingung).**

| Metrik            | $r = 0,5\text{ m}$ | $r = 1,5\text{ m}$ |
|-------------------|--------------------|--------------------|
| Erfolgsrate       | 10,4 %             | 69,7 %             |
| Terrain-Kollision | 15,9 %             | 9,4 %              |
| Timeout           | 73,7 %             | 27,7 %             |

## 5.4 Vergleich RL vs. RRT-Connect

Tabelle 5.4 stellt die Ergebnisse beider Verfahren gegenüber.

**Tabelle 5.4.: Vergleich von RL-Agent und RRT-Connect über beide Versuchsreihen ( $n = 1\,000$  je Bedingung).**

| Metrik                   | Phase 1 (Korridor) |         | Phase 2 (Weinberg) |         |
|--------------------------|--------------------|---------|--------------------|---------|
|                          | RL                 | RRT     | RL                 | RRT     |
| Erfolgsrate              | 71,0 %             | 82,0 %  | 10,4 %             | 85,7 %  |
| (relaxed $r = 1,5$ m)    | n. a.              |         | 69,7 %             | 98,4 %  |
| Hindernis-/Terrain-Koll. | 28,8 %             | 18,0 %  | 15,9 %             | 7,1 %   |
| Timeout                  | 0,0 %              | 0,0 %   | 73,7 %             | 7,9 %   |
| Landezeit (Median)       | 30,0 s             | 42,0 s  | 36,0 s             | 42,0 s  |
| Pfادهffizienz (Median)   | 1,19               | 1,09    | 1,07               | 0,99    |
| Min. Abstand (Mittel)    | 0,52 m             | 0,48 m  | 1,02 m             | 0,46 m  |
| Rechenzeit/Ep. (Median)  | 0,21 ms            | 80,4 ms | 0,25 ms            | 18,7 ms |

Der RRT-Planer übertrifft den RL-Agenten in der Erfolgsrate beider Umgebungen, wobei der Abstand im Weinberg deutlich größer ausfällt. Im Korridor dominieren bei beiden Verfahren Hinderniskollisionen; im Weinberg scheitert der RL-Agent überwiegend an Timeouts (73,7 %), während der RRT-Planer diese auf 7,9 % begrenzt. Der RL-Agent landet schneller (30,0 s gegenüber 42,0 s im Korridor) und benötigt deutlich weniger Rechenzeit (0,21 ms gegenüber 80,4 ms).<sup>1</sup> Dem steht ein einmaliger Trainingsaufwand von circa 24 Stunden gegenüber. Detaillierte Metriken mit Konfidenzintervallen finden sich in den Anhangstabellen B1 und D1.

<sup>1</sup>Einzelne Pfادهffizienzwerte knapp unter 1,0 sind ein Implementierungsartefakt: Das letzte Flugsegment wird aufgrund des automatischen Zurücksetzens der Umgebung nicht erfasst.

## 6 Diskussion

Dieses Kapitel interpretiert die Ergebnisse im Kontext der Nebenfragestellungen.

### 6.1 Leistungsfähigkeit in der Trainingsumgebung

#### 6.1.1 Trainingsverhalten und Konvergenz

Die zwei Phasen im Belohnungsverlauf lassen sich auf die Belohnungsstruktur zurückführen. In der Konvergenzphase (Iteration 0 bis 441) ermöglichen die dichten Komponenten ( $r_{\text{prog}}$ ,  $r_{\text{nah}}$ ,  $r_{\text{prox}}$ ) den initialen Anstieg; ab circa Iteration 200 setzt die Erfolgsbelohnung ein und beschleunigt den Anstieg. Ohne die dichten Komponenten würde dem Agenten das Gradientensignal für Teilfortschritte fehlen.

Der abrupte Kollaps ab Iteration 442 betrifft ausschließlich die Mittelwerte der Strategie bei nahezu unveränderter Standardabweichung ( $0,193 \rightarrow 0,194$ ), was auf eine katastrophale Strategieaktualisierung hindeutet. Obwohl der Clipping-Mechanismus von PPO (Schulman, Wolski u. a. 2017) große Strategieänderungen begrenzen soll, reicht er nicht aus, um solche Einbrüche zu verhindern. Eine mögliche Erklärung bieten Moalla u. a. (o. D.), die zeigen, dass eine schleichende Verschlechterung des Feature-Ranks den Clipping-Mechanismus unterlaufen kann. Eine schrittweise Reduktion der Lernrate verhinderte den Kollaps nicht; weitergehende Ansätze wie *Proximal Feature Optimization* (Moalla u. a. o. D.) wurden nicht evaluiert.

Die im Trainingsverlauf ansteigende Landezeit lässt sich auf die Belohnungsstruktur zurückführen.  $r_{\text{prog}} = d_t - d_{t+1}$  ist potentialbasiert im Sinne von Ng u. a. (1999) und verändert die optimale Strategie nicht.  $r_{\text{nah}} = \exp(-d_{t+1})$  hingegen ist nicht potentialbasiert: Jeder Zeitschritt in Zielnähe liefert Belohnung, unabhängig von weiterer Annäherung. Da es keine Zeitstrafe gibt, überwiegt der akkumulierte Schwebe-Anreiz. Dieses Phänomen dokumentieren auch Randlov und Alstrøm (1998): Ein nicht-potentialbasierter Shaping-Reward kann dazu führen, dass die akkumulierte Zwischenbelohnung die Zielbelohnung übersteigt.

#### 6.1.2 Dominante Fehlerart und Flugverhalten

Hinderniskollisionen (28,8 %) sind die einzig relevante Fehlerart. Mit oberflächenbasierter Kollisionserkennung (Abschnitt 5.2.1) steigt die Erfolgsrate auf 98,4 %. Die Diskrepanz erklärt sich aus drei Faktoren: Erstens ist die Hindernisproximität ( $w_{\text{prox}} = -0,2$ ) deutlich

schwächer gewichtet als Fortschritt und Nähe, sodass der Agent knappes Passieren rational in Kauf nimmt. Zweitens beschränkt das Entscheidungsintervall von 3 s die Reaktionszeit. Drittens erzwingt die nach unten gerichtete Kamera ein reaktives Flugverhalten, bei dem Hindernisse erst bei Annäherung erkannt werden. Das Flugmuster (direkter Abstieg, spätes Abbremsen, laterales Ausweichen mit mittlerem Minimalabstand von 0,52 m) ist präzise genug, um die tatsächliche Oberfläche zu verfehlen, durchdringt jedoch regelmäßig die konservative AABB-Hülle.

Zur Einordnung: Bartolomei u. a. (2022) berichten vergleichbare Raten (über 70 %) für ein RL-System mit Tiefenbildeingabe, allerdings für die Selektion sicherer Landezonen, nicht für die Navigation durch enge Hindernispassagen. Das Flugverhalten legt eine funktionale Rollenverteilung nahe: Die lateralen Ausweichmanöver setzen Hindernisinformation aus dem Tiefenbild voraus; der Zustandsvektor steuert Zielnavigation und Abstieg.

## **6.2 Transferleistung auf den Weinberg**

### **6.2.1 Leistungsabfall und Verschiebung der Fehlerarten**

Der Leistungsabfall von 71,0 % auf 10,4 % ist erwartbar. Aufschlussreich ist die Fehlerverschiebung: Während in Phase 1 Hinderniskollisionen dominieren, verschiebt sich die Verteilung in Phase 2 zu Zeitüberschreitungen (73,7 %) bei geringer Terrain-Kollisionsrate (15,9 %). Der Agent erreicht die Zielumgebung (mediane finale Distanz 1,19 m), kann dort jedoch nicht konvergieren. Mit erweitertem Erfolgswert von 1,5 m steigt die Erfolgsrate auf 69,7 %.

Die Ursache liegt in der strukturell veränderten Landesituation: Im Korridor ist die Landezone hindernisfrei; im Weinberg ragen Reihen direkt in die Landeumgebung. Dass die Aufgabe grundsätzlich lösbar ist, zeigt der RRT-Connect-Planer mit 85,7 % (vgl. Abschnitt 6.3).

### **6.2.2 Einordnung als Sim-to-Sim Transfer**

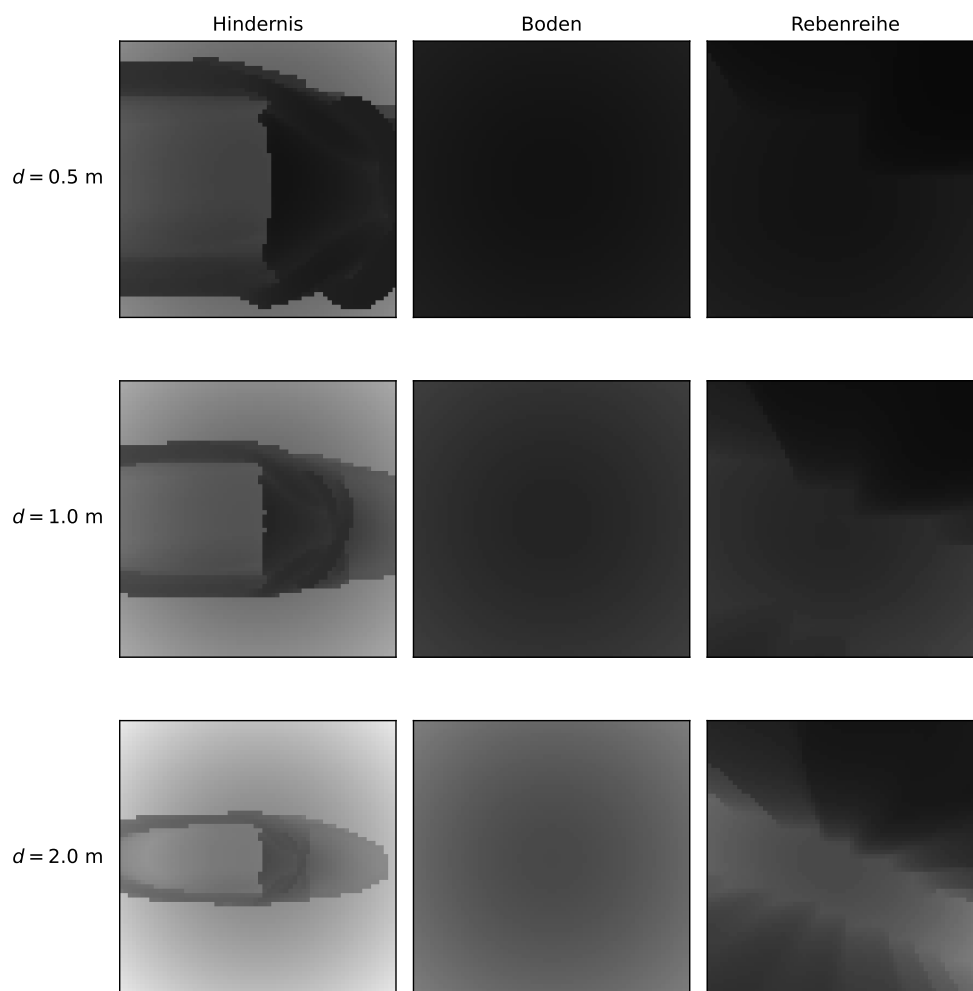
Dass die Navigationsleistung dennoch transferiert (69,7 % bei erweitertem Radius), lässt sich auf die umgebungsagnostische Beobachtungsstruktur zurückführen: Der Zustandsvektor kodiert relative Positionen, und das Tiefenbild enthält keine texturbasierten Merkmale. Sim-to-Real-Arbeiten berichten vergleichbare Einbußen: Polvara u. a. (2020) verlieren 28 Prozentpunkte, Ladosz u. a. (2024) 27 Prozentpunkte. Der hier beobachtete nominale Abfall von



61 Prozentpunkten relativiert sich bei erweitertem Radius auf 1,3 Prozentpunkte, was zeigt, dass der Umgebungswechsel die terminale Landephase betrifft, nicht die Navigation.

### 6.2.3 Perceptual Aliasing als Erklärung für die Terrain-Kollisionen

Ein möglicher Mechanismus hinter den 15,9 % Terrain-Kollisionen ist Perceptual Aliasing (Whitehead und Ballard 1991): Die Kamera erzeugt für Boden und Vegetation bei Annäherung ähnliche Signaturen (Abbildung 6.1). In der Trainingsumgebung ist diese Ambiguität unproblematisch, da die Landezone hindernisfrei ist. Im Weinberg bricht diese Strategie: Rebenreihen ragen in die Landezone, sodass ein Agent, der das Tiefenbild in Zielnähe zu ignorieren gelernt hat, mit Vegetation kollidiert.



**Abbildung 6.1.: Tiefenbilder der nach unten gerichteten Kamera bei drei Distanzen. Boden und Rebenreihe erzeugen deutlich ähnliche Signaturen als Korridorhindernisse.**

Verschärft wird das Problem durch die fehlende zeitliche Gedächtnisstruktur: Der Agent verarbeitet nur das aktuelle Tiefenbild. Laterales Ausweichen führt im Weinberg systematisch in die nächste Rebenreihe, da benachbarte Reihen außerhalb des Sichtfelds liegen. Singla u. a. (2018) zeigen, dass rekurrente Architekturen dieses Problem adressieren können.

### 6.3 Leistungslücke zum Pfadplaner

#### 6.3.1 Informationsasymmetrie und Erfolgsrate

Der RRT-Connect-Planer übertrifft den RL-Agenten in der Erfolgsrate: um 11 Prozentpunkte im Korridor und 75 Prozentpunkte im Weinberg. Dieser Vergleich quantifiziert den Preis minimaler Sensorik, nicht die algorithmische Überlegenheit des Planers. Unter oberflächenbasierter Kollisionserkennung erreicht der RL-Agent 98,4 % und liegt damit über der Planerleistung von 82,0 %. Dies bestätigt den von Semerikov u. a. (2025) beschriebenen Trend, dass lernbasierte Ansätze begrenzte Sensorausstattung algorithmisch kompensieren können.

#### 6.3.2 Landezeit und Rechenzeit

Der RL-Agent landet schneller (30,0 s gegenüber 42,0 s im Korridor), da er direkte Abstiegstrajektorien mit knappem Ausweichen lernt, während der Planer konservativere Pfade berechnet. Der deutlichste Unterschied zeigt sich in der Rechenzeit: Ein Forward-Pass benötigt 0,21 ms, RRT-Connect 80,4 ms pro Episode. Der RL-Agent entscheidet zudem lokal pro Zeitschritt, was ihn prinzipiell für dynamische Umgebungen geeignet macht.

### 6.4 Limitationen

Alle Ergebnisse basieren auf einem einzigen Trainingslauf (Seed); die Bootstrap-Konfidenzintervalle quantifizieren die Evaluations-, nicht die Trainingsvarianz. Die Belohnungsgewichte wurden manuell abgestimmt, wobei kleine Änderungen das Flugverhalten qualitativ verändern können. Die AABB-Kollisionserkennung überschätzt das Hindernisvolumen systematisch (71,0 % vs. 98,4 %); die oberflächenbasierte Variante war im Training aufgrund des Rechenaufwands nicht einsetzbar. Der Agent verarbeitet nur das aktuelle Tiefenbild ohne zeitliches Gedächtnis; der individuelle Beitrag beider Eingabemodalitäten wurde nicht durch eine Ablationsstudie quantifiziert. Im Training ist das Landeziel in allen Episoden identisch. Die Simulation bietet perfekte Sensorik ohne Rauschen und keine Motorverzögerung; Ergebnisse sind nicht direkt auf reale Drohnen übertragbar.

## 7 Fazit und Ausblick

### 7.1 Beantwortung der Forschungsfragen

Die vorliegende Arbeit untersuchte, inwiefern ein RL-basierter Ansatz eine autonome Drohne befähigen kann, mithilfe lokaler Tiefensensorik in hindernisreichen Umgebungen sicher zu landen.

Der trainierte Agent erlernt eine reaktive Ausweichstrategie, deren Leistung primär durch strukturelle Entwurfsentscheidungen begrenzt wird: Sensorhorizont, Entscheidungsintervall und Belohnungsgewichtung bestimmen, wie knapp der Agent Hindernisse passiert. Die Wahl der Kollisionsmetrik erweist sich als entscheidender Faktor: Unter oberflächenbasierter Erkennung erreicht der Agent 98,4 % und übertrifft damit den privilegierten Pfadplaner (82,0 %).

Die Transferuntersuchung zeigt eine klare Trennung zweier Teilfähigkeiten: Hindernisnavigation generalisiert auf die Weinbergumgebung (69,7 % bei erweitertem Erfolgsradius); die terminale Landephase scheitert an der im Training nie erfahrenen Konfiguration von Vegetation in der Landezone, verstärkt durch Perceptual Aliasing und fehlendes zeitliches Gedächtnis.

Der Vergleich mit dem privilegierten Pfadplaner quantifiziert den Vorteil vollständiger Umgebungskennntnis. Unter fairer Kollisionsmetrik löst der lernbasierte Ansatz das Navigationsproblem im Korridor erfolgreicher als der Planer, bei um Größenordnungen niedrigerer Inferenzzeit. Im Weinberg kehrt sich dieses Verhältnis um; die Leistungslücke reflektiert die begrenzte Trainingsdiversität.

Insgesamt zeigt die Arbeit, dass ein RL-Agent mit minimaler Sensorik die kombinierte Aufgabe aus Zielnavigation und Hindernisvermeidung in Simulation lösen kann und die erlernte Strategie sich teilweise auf ein realistisches Szenario übertragen lässt.

### 7.2 Perspektiven für zukünftige Forschung

Eine Multi-Seed-Validierung und eine Skalierung auf größere Trainingsbatches könnten die statistische Belastbarkeit und die Strategiequalität verbessern. Eine rekurrente Architektur oder ein Tiefenbild-Stapel würde dem Agenten zeitliches Gedächtnis verleihen und das Perceptual-Aliasing-Problem adressieren. Eine Ablationsstudie könnte den individuellen Beitrag von Tiefenbild und Zustandsvektor quantifizieren. Domain Randomization, insbe-

---

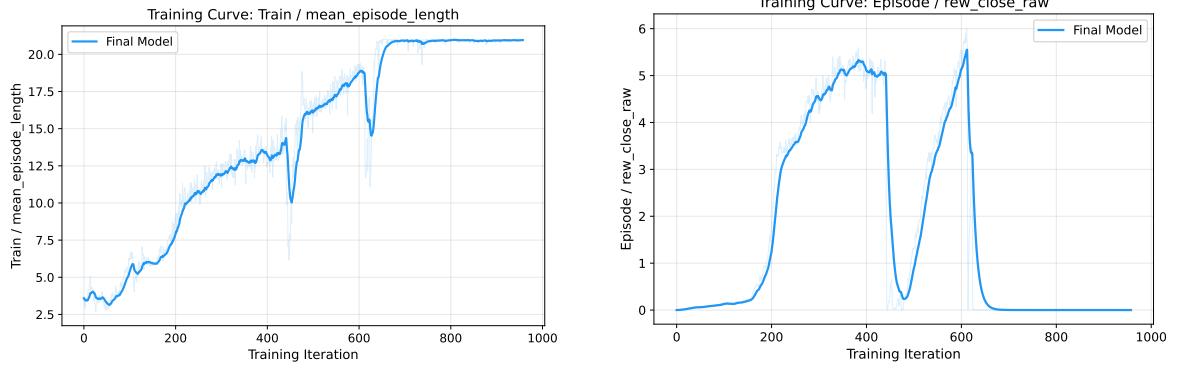
sondere Sensorrauschen und variable Zielpositionen, könnte die Robustheit und den Transfer auf reale Hardware vorbereiten. Langfristig stellt der Sim-to-Real-Transfer den entscheidenden nächsten Schritt dar, um die gezeigte Leistungsfähigkeit in realen Einsatzszenarien zu validieren.

## Anhang

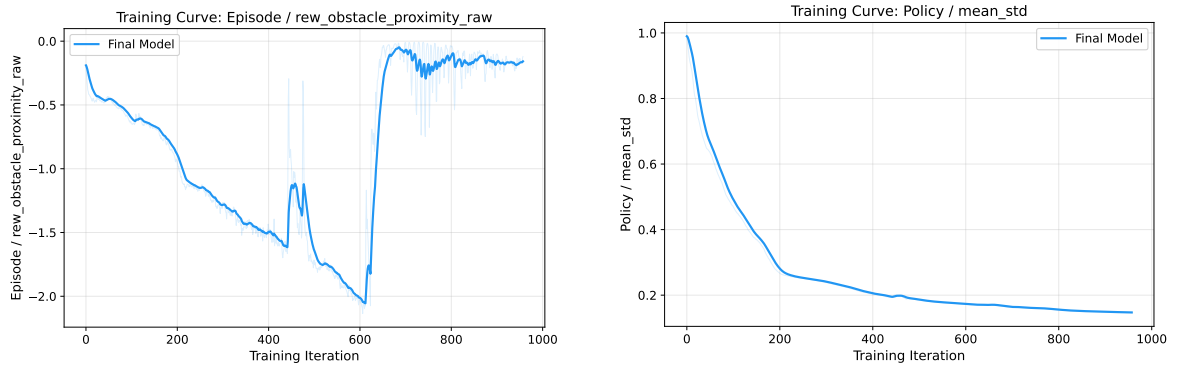
### Anhangsverzeichnis

|          |                                                                                      |      |
|----------|--------------------------------------------------------------------------------------|------|
| Anhang A | Ergänzende Trainingsverläufe . . . . .                                               | VIII |
| Anhang B | Phase-1-Evaluationsmetriken . . . . .                                                | IX   |
| Anhang C | Phase-1-Evaluationsmetriken (oberflächenbasierte Kollisionserken-<br>nung) . . . . . | X    |
| Anhang D | Phase-2-Evaluationsmetriken . . . . .                                                | XI   |
| Anhang E | PID-Reglerparameter . . . . .                                                        | XII  |

## A Ergänzende Trainingsverläufe



**Abbildung A1.:** Mittlere Episodenlänge (links) und Näherungsbelohnung (rechts). Die Episodenlänge gibt die Anzahl der Entscheidungsschritte pro Episode an. Die Näherungsbelohnung steigt mit abnehmender Distanz zum Ziel. Geglättet mit EMA  $\alpha = 0,9$ .



**Abbildung A2.:** Hindernisabstandsstrafe (links) und mittlere Standardabweichung der Aktionsverteilung (rechts). Geglättet mit EMA  $\alpha = 0,9$ .

## B Phase-1-Evaluationsmetriken

**Tabelle B1.: Phase-1-Ergebnisse des RL-Agenten (1 000 Episoden, Checkpoint 400, ungesehene Hindernisse). Konfidenzintervalle: 95 %-Bootstrap (Perzentilmethode,  $B = 10\,000$ ). Landezeit, Pfadeffizienz und Rechenzeit beziehen sich ausschließlich auf erfolgreiche Episoden ( $n = 710$ ).**

| Metrik                          | Wert         | 95 %-KI / IQR    |
|---------------------------------|--------------|------------------|
| Erfolgsrate                     | 71,0 %       | [68,2; 73,8]     |
| Hinderniskollision              | 28,8 % (288) | [26,0; 31,6]     |
| Verlassen des Korridors         | 0,2 % (2)    | [0,0; 0,5]       |
| Bodenkollision                  | 0,0 % (0)    | —                |
| Timeout                         | 0,0 % (0)    | —                |
| Landezeit (Median)              | 30,0 s       | IQR [27,0; 36,0] |
| Pfadeffizienz (Median)          | 1,19         | IQR [1,13; 1,25] |
| Min. Hindernisabstand (Mittel)  | 0,52 m       | [0,50; 0,54]     |
| Min. Hindernisabstand (global)* | 0,00 m       | —                |
| Rechenzeit/Episode (Median)     | 0,21 ms      | IQR [0,19; 0,25] |

\* Stammt aus Episoden, die mit Kollision endeten (Distanz = 0 bei Kontakt).

## C Phase-1-Evaluationsmetriken (oberflächenbasierte Kollisionserkennung)

**Tabelle C1.: Phase-1-Ergebnisse des RL-Agenten mit oberflächenbasierter Kollisionserkennung (1 000 Episoden, Checkpoint 400, ungesehene Hindernisse). Konfidenzintervalle: 95 %-Bootstrap (Perzentilmethode,  $B = 10\,000$ ). Landezeit, Pfadeffizienz und Rechenzeit beziehen sich ausschließlich auf erfolgreiche Episoden ( $n = 984$ ).**

| Metrik                          | Wert       | 95 %-KI / IQR    |
|---------------------------------|------------|------------------|
| Erfolgsrate                     | 98,4 %     | [97,6; 99,1]     |
| Hinderniskollision              | 1,2 % (12) | [0,6; 1,9]       |
| Verlassen des Korridors         | 0,2 % (2)  | [0,0; 0,5]       |
| Bodenkollision                  | 0,0 % (0)  | —                |
| Timeout                         | 0,2 % (2)  | [0,0; 0,5]       |
| Landezeit (Median)              | 33,0 s     | IQR [30,0; 36,0] |
| Pfadeffizienz (Median)          | 1,19       | IQR [1,13; 1,25] |
| Min. Hindernisabstand (Mittel)  | 0,51 m     | [0,49; 0,53]     |
| Min. Hindernisabstand (global)* | 0,00 m     | —                |
| Rechenzeit/Episode (Median)     | 0,23 ms    | IQR [0,21; 0,25] |

\*Stammt aus Episoden, die mit Kollision endeten (Distanz = 0 bei Kontakt).



## D Phase-2-Evaluationsmetriken

**Tabelle D1.: Phase-2-Ergebnisse des RL-Agenten (1 000 Episoden, Checkpoint 400, Weinberg-Transfer, Erfolgsradius 0,5 m). Konfidenzintervalle: 95 %-Bootstrap (Perzentilmethode,  $B = 10\,000$ ). Landezeit, Pfadeffizienz und Rechenzeit beziehen sich ausschließlich auf erfolgreiche Episoden ( $n = 104$ ).**

| Metrik                         | Wert                                         | 95 %-KI / IQR    |
|--------------------------------|----------------------------------------------|------------------|
| Erfolgsrate                    | 10,4 %                                       | [8,5; 12,4]      |
| Terrain-Kollision              | 15,9 % (159)                                 | [13,6; 18,2]     |
| Timeout                        | 73,7 % (737)                                 | [71,0; 76,4]     |
| Bodenkollision                 | entfällt (kein fester Bodenkollisions-Check) |                  |
| Verlassen des Korridors        | entfällt (kein Korridorbegrenzungsrahmen)    |                  |
| Landezeit (Median)             | 36,0 s                                       | IQR [33,0; 36,0] |
| Pfadeffizienz (Median)         | 1,07                                         | IQR [1,05; 1,09] |
| Finale Zieldistanz (Median)    | 1,19 m                                       | —                |
| Min. Terrain-Abstand (Mittel)  | 1,02 m                                       | [0,98; 1,06]     |
| Min. Terrain-Abstand (global)* | 0,00 m                                       | —                |
| Rechenzeit/Episode (Median)    | 0,25 ms                                      | IQR [0,23; 0,27] |

\* Stammt aus Episoden, die mit Terrain-Kollision endeten (Distanz = 0 bei Kontakt).

## E PID-Reglerparameter

**Tabelle E1.: PID-Verstärkungen des kaskadierenden Reglers.**

| Regelkreis            | $K_p$ | $K_i$ | $K_d$ |
|-----------------------|-------|-------|-------|
| Position $x/y$        | 1,0   | 0     | 0,7   |
| Position $z$          | 1,5   | 0     | 1,0   |
| Geschwindigkeit $x/y$ | 16,0  | 0     | 8,0   |
| Geschwindigkeit $z$   | 100,0 | 2,0   | 10,0  |
| Roll / Nick           | 6,0   | 0     | 3,0   |
| Gier                  | 0,5   | 0     | 0,8   |

## Literatur

- Amendola, José, Linga Reddy Cenkeramaddi und Ajit Jha (2024). „Drone Landing and Reinforcement Learning: State-of-Art, Challenges and Opportunities“. In: *IEEE Open Journal of Intelligent Transportation Systems* 5, S. 520–539. ISSN: 2687-7813. DOI: 10.1109/OJITS.2024.3444487. URL: <https://ieeexplore.ieee.org/document/10637701> (besucht am 18.12.2025).
- Arafat, Muhammad Yeasir, Muhammad Morshed Alam und Sangman Moh (27. Jan. 2023). „Vision-Based Navigation Techniques for Unmanned Aerial Vehicles: Review and Challenges“. In: *Drones* 7.2. ISSN: 2504-446X. DOI: 10.3390/drones7020089. URL: <https://www.mdpi.com/2504-446X/7/2/89> (besucht am 13.05.2026).
- Bartolomei, Luca, Yves Kompis, Lucas Teixeira und Margarita Chli (Okt. 2022). „Autonomous Emergency Landing for Multicopters Using Deep Reinforcement Learning“. In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), S. 3392–3399. DOI: 10.1109/IROS47612.2022.9981152. URL: <https://ieeexplore.ieee.org/document/9981152> (besucht am 18.12.2025).
- Basso, Bruno und John Antle (Apr. 2020). „Digital Agriculture to Design Sustainable Agricultural Systems“. In: *Nature Sustainability* 3.4, S. 254–256. ISSN: 2398-9629. DOI: 10.1038/s41893-020-0510-0. URL: <https://www.nature.com/articles/s41893-020-0510-0> (besucht am 03.05.2026).
- Botsch, Benny (2023). „Parametrische Methoden“. In: *Maschinelles Lernen - Grundlagen und Anwendungen: Mit Beispielen in Python*. Hrsg. von Benny Botsch. Berlin, Heidelberg: Springer, S. 61–102. ISBN: 978-3-662-67277-8. DOI: 10.1007/978-3-662-67277-8\_5. URL: [https://doi.org/10.1007/978-3-662-67277-8\\_5](https://doi.org/10.1007/978-3-662-67277-8_5) (besucht am 06.06.2026).
- Calvin, Katherine u. a. (25. Juli 2023). *IPCC, 2023: Climate Change 2023: Synthesis Report. Contribution of Working Groups I, II and III to the Sixth Assessment Report of the*

- Intergovernmental Panel on Climate Change [Core Writing Team, H. Lee and J. Romero (Eds.)]. IPCC, Geneva, Switzerland. Intergovernmental Panel on Climate Change (IPCC). DOI: 10.59327/IPCC/AR6-9789291691647. URL: <https://www.ipcc.ch/report/ar6/syr/> (besucht am 01.04.2026).*
- Climate Change Trends and Impacts on California Agriculture: A Detailed Review | MDPI (2026). URL: <https://www.mdpi.com/2073-4395/8/3/25#Conclusions> (besucht am 01.04.2026).*
- Cobbe, Karl, Jacob Hilton, Oleg Klimov und John Schulman (2021). „Phasic Policy Gradient“. In: *Proceedings of the 38th International Conference on Machine Learning*. Bd. 139. Proceedings of Machine Learning Research. PMLR, S. 2020–2027.
- Coumans, Erwin und Yunfei Bai (2016–2021). *PyBullet, a Python Module for Physics Simulation for Games, Robotics and Machine Learning*. URL: <http://pybullet.org> (besucht am 20.05.2026).
- Downs, Laura, Anthony Francis, Nate Koenig, Brandon Kinman, Ryan Hickman, Krista Reymann, Thomas B. McHugh und Vincent Vanhoucke (25. Apr. 2022). *Google Scanned Objects: A High-Quality Dataset of 3D Scanned Household Items*. DOI: 10.48550/arXiv.2204.11918. arXiv: 2204.11918 [cs.RO]. URL: <http://arxiv.org/abs/2204.11918> (besucht am 23.05.2026). Vorveröffentlichung.
- Duckett, Tom u. a. (2. Aug. 2018). *Agricultural Robotics: The Future of Robotic Agriculture*. DOI: 10.48550/arXiv.1806.06762. arXiv: 1806.06762 [cs]. URL: <http://arxiv.org/abs/1806.06762> (besucht am 03.05.2026). Vorveröffentlichung.
- Efron, Bradley und Robert J. Tibshirani (1994). *An Introduction to the Bootstrap*. noch lesen. New York: Chapman & Hall/CRC. ISBN: 978-0-412-04231-7.
- Eßer, Julian, Gabriel B Margolis, Oliver Urbann, Soren Kerner und Pulkit Agrawal (o. D.). „Action Space Design in Reinforcement Learning for Robot Motor Skills“. In: ().

- Food and Agriculture Organization of the United Nations, Hrsg. (2017). *The Future of Food and Agriculture: Trends and Challenges*. Rome: Food and Agriculture Organization of the United Nations. 163 S. ISBN: 978-92-5-109551-5.
- Genesis Authors (Dez. 2024). *Genesis: A Generative and Universal Physics Engine for Robotics and Beyond*. URL: <https://github.com/Genesis-Embodied-AI/Genesis> (besucht am 20.05.2026).
- Guebsi, Ridha, Sonia Mami und Karem Chokmani (2024). „Drones in Precision Agriculture: A Comprehensive Review of Applications, Technologies, and Challenges“. In: *Drones* 8.11, S. 686. DOI: 10.3390/drones8110686.
- Haarnoja, Tuomas, Aurick Zhou, Pieter Abbeel und Sergey Levine (2018). „Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor“. In: *Proceedings of the 35th International Conference on Machine Learning*. Bd. 80. Proceedings of Machine Learning Research. PMLR, S. 1861–1870. URL: <https://proceedings.mlr.press/v80/haarnoja18b.html>.
- Hodge, Victoria J., Richard Hawkins und Rob Alexander (1. März 2021). „Deep Reinforcement Learning for Drone Navigation Using Sensor Data“. In: *Neural Computing and Applications* 33.6, S. 2015–2033. ISSN: 1433-3058. DOI: 10.1007/s00521-020-05097-x. URL: <https://doi.org/10.1007/s00521-020-05097-x> (besucht am 15.04.2026).
- Hultgren, Andrew, Tamma Carleton, Michael Delgado, Diana R. Gergel, Michael Greenstone, Trevor Houser, Solomon Hsiang, Amir Jina, Robert E. Kopp, Steven B. Malevich, Kelly E. McCusker, Terin Mayer, Ishan Nath, James Rising, Ashwin Rode und Jiacan Yuan (Juni 2025). „Impacts of Climate Change on Global Agriculture Accounting for Adaptation“. In: *Nature* 642.8068, S. 644–652. ISSN: 1476-4687. DOI: 10.1038/s41586-025-09085-w. URL: <https://www.nature.com/articles/s41586-025-09085-w> (besucht am 01.04.2026).
- Jarraya, Imen, Abdulrahman Al-Batati, Muhammad Bilal Kadri, Mohamed Abdelkader, Adel Ammar, Wadii Boulila und Anis Koubaa (7. Apr. 2025). „Gnss-Denied Unmanned Aerial Vehicle Navigation: Analyzing Computational Complexity, Sensor Fusion, and Localization Methodologies“. In: *Satellite Navigation* 6.1, S. 9. ISSN: 2662-1363. DOI:

10.1186/s43020-025-00162-z. URL: <https://doi.org/10.1186/s43020-025-00162-z> (besucht am 14.05.2026).

Kanellakis, Christoforos und George Nikolakopoulos (1. Juli 2017). „Survey on Computer Vision for UAVs: Current Developments and Trends“. In: *Journal of Intelligent & Robotic Systems* 87.1, S. 141–168. ISSN: 1573-0409. DOI: 10.1007/s10846-017-0483-z. URL: <https://doi.org/10.1007/s10846-017-0483-z> (besucht am 13.05.2026).

Kaufmann, Elia, Leonard Bauersfeld, Antonio Loquercio, Matthias Müller, Vladlen Koltun und Davide Scaramuzza (Aug. 2023). „Champion-Level Drone Racing Using Deep Reinforcement Learning“. In: *Nature* 620.7976, S. 982–987. ISSN: 1476-4687. DOI: 10.1038/s41586-023-06419-4. URL: <https://www.nature.com/articles/s41586-023-06419-4> (besucht am 18.12.2025).

Kaufmann, Elia, Leonard Bauersfeld und Davide Scaramuzza (22. Feb. 2022). *A Benchmark Comparison of Learned Control Policies for Agile Quadrotor Flight*. arXiv.org. URL: <https://arxiv.org/abs/2202.10796v1> (besucht am 03.06.2026).

Kim, Minwoo, Jongyun Kim, Minjae Jung und Hyondong Oh (15. Juli 2022). „Towards Monocular Vision-Based Autonomous Flight through Deep Reinforcement Learning“. In: *Expert Systems with Applications* 198, S. 116742. ISSN: 0957-4174. DOI: 10.1016/j.eswa.2022.116742. URL: <https://www.sciencedirect.com/science/article/pii/S0957417422002111> (besucht am 16.05.2026).

Koenig, Nathan und Andrew Howard (2004). „Design and Use Paradigms for Gazebo, an Open-Source Multi-Robot Simulator“. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Bd. 3, S. 2149–2154. DOI: 10.1109/IROS.2004.1389727.

Ladosz, Pawel, Meraj Mammadov, Heejung Shin, Woojae Shin und Hyondong Oh (Mai 2024). „Autonomous Landing on a Moving Platform Using Vision-Based Deep Reinforcement Learning“. In: *IEEE Robotics and Automation Letters* 9.5, S. 4575–4582. ISSN: 2377-3766. DOI: 10.1109/LRA.2024.3379837. URL: <https://ieeexplore.ieee.org/document/10476692/> (besucht am 14.05.2026).

- LeCun, Yann, Yoshua Bengio und Geoffrey Hinton (Mai 2015). „Deep Learning“. In: *Nature* 521.7553, S. 436–444. ISSN: 1476-4687. DOI: 10.1038/nature14539. URL: <https://www.nature.com/articles/nature14539> (besucht am 06.06.2026).
- Loquercio, Antonio, Elia Kaufmann, René Ranftl, Matthias Müller, Vladlen Koltun und Davide Scaramuzza (6. Okt. 2021). „Learning High-Speed Flight in the Wild“. In: *Science Robotics* 6.59, eabg5810. DOI: 10.1126/scirobotics.abg5810. URL: <https://www.science.org/doi/10.1126/scirobotics.abg5810> (besucht am 03.05.2026).
- Makoviychuk, Viktor, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa und Gavriel State (2021). „Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning“. In: *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*. URL: [https://openreview.net/forum?id=fgFBtYgJQX\\_](https://openreview.net/forum?id=fgFBtYgJQX_).
- Martini, Mauro, Simone Cerrato, Francesco Salvetti, Simone Angarano und Marcello Chiaberge (Aug. 2022). „Position-Agnostic Autonomous Navigation in Vineyards with Deep Reinforcement Learning“. In: *2022 IEEE 18th International Conference on Automation Science and Engineering (CASE)*. 2022 IEEE 18th International Conference on Automation Science and Engineering (CASE), S. 477–484. DOI: 10.1109/CASE49997.2022.9926582. URL: <https://ieeexplore.ieee.org/abstract/document/9926582> (besucht am 03.06.2026).
- Mashori, Abdul Sattar, Fuzhong Li, Muhammad Aman, Wuping Zhang, Shujie Jia, Aamir Ali, Nida Jabeen, Wangli Hao und Yuqiao Yan (1. Aug. 2026). „Remote Sensing through UAVs for Precision Agriculture: Applications, Technical Foundations, Current Barriers, and Future Opportunities“. In: *Smart Agricultural Technology* 14, S. 102074. ISSN: 2772-3755. DOI: 10.1016/j.atech.2026.102074. URL: <https://www.sciencedirect.com/science/article/pii/S2772375526002959> (besucht am 03.05.2026).

- Meier, Lorenz, Dominik Honegger und Marc Pollefeys (Mai 2015). „PX4: A Node-Based Multithreaded Open Source Robotics Framework for Deeply Embedded Platforms“. In: *2015 IEEE International Conference on Robotics and Automation (ICRA)*. 2015 IEEE International Conference on Robotics and Automation (ICRA). Seattle, WA, USA: IEEE, S. 6235–6240. ISBN: 978-1-4799-6923-4. DOI: 10.1109/ICRA.2015.7140074. URL: <http://ieeexplore.ieee.org/document/7140074/> (besucht am 14.05.2026).
- Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg und Demis Hassabis (Feb. 2015). „Human-Level Control through Deep Reinforcement Learning“. In: *Nature* 518.7540, S. 529–533. ISSN: 1476-4687. DOI: 10.1038/nature14236. URL: <https://www.nature.com/articles/nature14236> (besucht am 03.05.2026).
- Moalla, Skander, Andrea Miele, Daniil Pyatko, Razvan Pascanu und Caglar Gulcehre (o. D.). „No Representation, No Trust: Connecting Representation, Collapse, and Trust Issues in PPO“. In: ().
- Ng, Andrew Y., Daishi Harada und Stuart J. Russell (27. Juni 1999). „Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping“. In: *Proceedings of the Sixteenth International Conference on Machine Learning*. ICML '99. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., S. 278–287. ISBN: 978-1-55860-612-8.
- Patrik, Aurello, Gaudi Utama, Alexander Agung Santoso Gunawan, Andry Chowanda, Jarot S. Suroso, Rizatus Shofiyanti und Widodo Budiharto (Dez. 2019). „GNSS-based Navigation Systems of Autonomous Drone for Delivering Items“. In: *Journal of Big Data* 6.1, S. 53. ISSN: 2196-1115. DOI: 10.1186/s40537-019-0214-3. URL: <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-019-0214-3> (besucht am 13.05.2026).
- Peter, Robinroy, Lavanya Ratnabala, Demetros Aschu, Aleksey Fedoseev und Dzmitry Tsetserukou (25. Juni 2024). *TornadoDrone: Bio-inspired DRL-based Drone Landing on*



- 6D Platform with Wind Force Disturbances*. Version 2. DOI: 10.48550/arXiv.2406.16164. arXiv: 2406.16164 [cs]. URL: <http://arxiv.org/abs/2406.16164> (besucht am 07.03.2026). Vorveröffentlichung.
- Polvara, Riccardo, Massimiliano Patacchiola, Marc Hanheide und Gerhard Neumann (25. Feb. 2020). „Sim-to-Real Quadrotor Landing via Sequential Deep Q-Networks and Domain Randomization“. In: *Robotics* 9.1. ISSN: 2218-6581. DOI: 10.3390/robotics9010008. URL: <https://www.mdpi.com/2218-6581/9/1/8> (besucht am 07.02.2026).
- Precision Landing* (2026). PX4 Autopilot. URL: [https://docs.px4.io/main/en/advanced\\_features/precland](https://docs.px4.io/main/en/advanced_features/precland) (besucht am 14.05.2026).
- Radoglou-Grammatikis, Panagiotis, Panagiotis Sarigiannidis, Thomas Lagkas und Ioannis Moscholios (8. Mai 2020). „A Compilation of UAV Applications for Precision Agriculture“. In: *Computer Networks* 172, S. 107148. ISSN: 1389-1286. DOI: 10.1016/j.comnet.2020.107148. URL: <https://www.sciencedirect.com/science/article/pii/S138912862030116X> (besucht am 03.05.2026).
- Randlov, Jette und Preben Alstrøm (1. Jan. 1998). *Learning to Drive a Bicycle Using Reinforcement Learning and Shaping*. S. 471. 463 S.
- Rudin, Nikita, David Hoeller, Philipp Reist und Marco Hutter (2022). „Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning“. In: *Proceedings of the 5th Conference on Robot Learning*. Bd. 164. Proceedings of Machine Learning Research. PMLR, S. 91–100.
- Schulman, John, Sergey Levine, Philipp Moritz, Michael I. Jordan und Pieter Abbeel (2015). „Trust Region Policy Optimization“. In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. Bd. 37. Proceedings of Machine Learning Research. PMLR, S. 1889–1897. URL: <https://proceedings.mlr.press/v37/schulman15.html>.
- Schulman, John, Philipp Moritz, Sergey Levine, Michael I. Jordan und Pieter Abbeel (2016). „High-Dimensional Continuous Control Using Generalized Advantage Estimation“. In: *4th International Conference on Learning Representations (ICLR)*. URL: <https://arxiv.org/abs/1506.02438>.

- Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford und Oleg Klimov (28. Aug. 2017). *Proximal Policy Optimization Algorithms*. arXiv: 1707.06347 [cs.LG]. URL: <https://arxiv.org/abs/1707.06347>.
- Semerikov, Serhiy O., Pavlo P. Nechypurenko, Tetiana A. Vakaliuk, Iryna S. Mintii und Andrii O. Kolhatin (28. Okt. 2025). „Vision-Based Autonomous UAV Landing: A Comprehensive Review of Technologies, Techniques, and Applications“. In: *Journal of Intelligent & Robotic Systems* 111.4, S. 115. ISSN: 1573-0409. DOI: 10.1007/s10846-025-02314-4. URL: <https://doi.org/10.1007/s10846-025-02314-4> (besucht am 05.05.2026).
- Singla, Abhik, Sindhu Padakandla und Shalabh Bhatnagar (8. Nov. 2018). *Memory-Based Deep Reinforcement Learning for Obstacle Avoidance in UAV with Limited Environment Knowledge*. arXiv.org. URL: <https://arxiv.org/abs/1811.03307v1> (besucht am 30.05.2026).
- Sutton, Richard S. und Andrew Barto (2020). *Reinforcement Learning: An Introduction*. Second edition. Adaptive Computation and Machine Learning. Cambridge, Massachusetts London, England: The MIT Press. 526 S. ISBN: 978-0-262-03924-6.
- Tavasci, Luca, Francesco Nex und Stefano Gandolfi (20. Sep. 2024). „Reliability of Real-Time Kinematic (RTK) Positioning for Low-Cost Drones' Navigation across Global Navigation Satellite System (GNSS) Critical Environments“. In: *Sensors* 24.18. ISSN: 1424-8220. DOI: 10.3390/s24186096. URL: <https://www.mdpi.com/1424-8220/24/18/6096> (besucht am 14.05.2026).
- Tayar, Marco S., Lucas K. de Oliveira, Felipe Andrade G. Tommaselli, Juliano D. Negri, Thiago H. Segreto, Ricardo V. Godoy und Marcelo Becker (22. Aug. 2025). *Autonomous UAV Flight Navigation in Confined Spaces: A Reinforcement Learning Approach*. arXiv.org. DOI: 10.1109/LARS69345.2025.11273007. URL: <https://arxiv.org/abs/2508.16807v2> (besucht am 03.06.2026).
- Unde, Sanket S., V. K. Kurkute, Sachin S. Chavan, Dadaso D. Mohite, Akshay A. Harale und Ayaan Chougale (16. Sep. 2025). „The Expanding Role of Multirotor UAVs in Precision Agriculture with Applications AI Integration and Future Prospects“. In: *Discover Mechanical Engineering* 4.1, S. 38. ISSN: 2731-6564. DOI: 10.1007/s44245-

025 – 00132 – 4. URL: <https://doi.org/10.1007/s44245-025-00132-4> (besucht am 29.01.2026).

Voos, Holger und Haitham Bou-Ammar (Sep. 2010). „Nonlinear Tracking and Landing Controller for Quadrotor Aerial Robots“. In: *2010 IEEE International Conference on Control Applications*. Control (MSC). Yokohama, Japan: IEEE, S. 2136–2141. ISBN: 978-1-4244-5362-7. DOI: 10.1109/CCA.2010.5611204. URL: <http://ieeexplore.ieee.org/document/5611204/> (besucht am 13.05.2026).

Wang, Junqiao, Zhongliang Yu, Dong Zhou, Jiaqi Shi und Runran Deng (9. Dez. 2024). *Vision-Based Deep Reinforcement Learning of UAV Autonomous Navigation Using Privileged Information*. Version 1. DOI: 10.48550/arXiv.2412.06313. arXiv: 2412.06313 [cs]. URL: <http://arxiv.org/abs/2412.06313> (besucht am 03.04.2026). Vorveröffentlichung.

Wei, Peng, Prabhash Ragbir, Stavros G. Vougioukas und Zhaodan Kong (1. Nov. 2025). „Vision-Based Navigation of Unmanned Aerial Vehicles in Orchards: An Imitation Learning Approach“. In: *Computers and Electronics in Agriculture* 238, S. 110802. ISSN: 0168-1699. DOI: 10.1016/j.compag.2025.110802. URL: <https://www.sciencedirect.com/science/article/pii/S0168169925009081> (besucht am 03.06.2026).

Whitehead, Steven D. und Dana H. Ballard (1. Juli 1991). „Learning to Perceive and Act by Trial and Error“. In: *Machine Learning* 7.1, S. 45–83. ISSN: 1573-0565. DOI: 10.1023/A:1022619109594. URL: <https://doi.org/10.1023/A:1022619109594> (besucht am 30.05.2026).

*World Population Prospects* (2026). URL: <https://population.un.org/wpp/> (besucht am 03.05.2026).

Wubben, Jamie, Francisco Fabra, Carlos T. Calafate, Tomasz Krzeszowski, Johann M. Marquez-Barja, Juan-Carlos Cano und Pietro Manzoni (12. Dez. 2019). „Accurate Landing of Unmanned Aerial Vehicles Using Ground Pattern Recognition“. In: *Electronics* 8.12. ISSN: 2079-9292. DOI: 10.3390/electronics8121532. URL: <https://www.mdpi.com/2079-9292/8/12/1532> (besucht am 14.05.2026).

- Xu, Zhefan, Xinming Han, Haoyu Shen, Hanyu Jin und Kenji Shimada (24. Sep. 2024). „NavRL: Learning Safe Flight in Dynamic Environments“. In: *arXiv preprint*. DOI: 10.48550/arXiv.2409.15634. arXiv: 2409.15634 [cs.RO].
- Yang, Tao, Peiqi Li, Huiming Zhang, Jing Li und Zhi Li (15. Mai 2018). „Monocular Vision SLAM-Based UAV Autonomous Landing in Emergencies and Unknown Environments“. In: *Electronics* 7.5. ISSN: 2079-9292. DOI: 10.3390/electronics7050073. URL: <https://www.mdpi.com/2079-9292/7/5/73> (besucht am 07.02.2026).

**Erklärung**

Ich versichere, dass ich die vorliegende Arbeit mit dem Thema:

*„Titel“*

selbständig und nur unter Verwendung der angegebenen Quellen und Hilfsmittel angefertigt habe, insbesondere sind wörtliche oder sinngemäße Zitate als solche gekennzeichnet. Mir ist bekannt, dass Zuwiderhandlung auch nachträglich zur Aberkennung des Abschlusses führen kann. Ich versichere, dass das elektronische Exemplar mit den gedruckten Exemplaren übereinstimmt.

Leipzig, den Datum

---

AUTORENNAME